

中國人民大學

专业硕士学位论文

面向智能体的大模型自进化强化学习系统研
(中文题目) 究与实现

Research and Implementation of
a Self-Evolving Reinforcement
Learning System for Agentic

(英文题目) Large Language Models

作者学号: 2025104468

作者姓名: 孙豪

所在学院: 智慧治理学院

专业名称: 大语言模型、强化学习

导师姓名: 卞昊穹 孙殿森
自进化强化学习; 多智能体合成

论文主题词: 数据; 差分驱动奖励

论文提交日期: 2027 年 04 月

独创性声明

本人郑重声明：所呈交的论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得中国人民大学或其他教育机构的学位或证书所使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

论文作者（签名）：_____ 日 期：_____

关于论文使用授权的说明

本人完全了解中国人民大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

论文作者（签名）：_____ 日 期：_____

指导老师（签名）：_____ 日 期：_____

授权书影印件

摘要

大语言模型驱动的智能体在代码沙箱、文件系统等真实环境中通过多步工具调用完成任务，强化学习是让其持续变强的主流手段。然而，交互式智能体的强化学习面临一个根本性瓶颈：**训练数据与可信奖励均难以靠人工规模化获得**。真实用户回流数据积累慢、覆盖窄；而奖励若仅依赖智能体自述，模型会迅速学会谎报完成这一奖励作弊行为。

本文设计并实现一套面向智能体的自进化强化学习系统，其核心是让智能体在无人工标注条件下**自主产生训练数据、自主获得可信奖励、自主生成下一轮训练任务**。系统以**自进化**为第一目标：除初始种子外，后续所有训练查询由系统基于上一轮执行经验自主生成——观察者用大模型概括上轮最优轨迹的执行情况，提问者据此生成符合人设的新种子，沙箱文件系统状态跨步继承到新环境，使智能体在上轮产出的基础上继续工作。超采样-淘汰-选组机制进一步保证每步训练只用梯度最优的轨迹子集，提升自进化效率。

在数据自主产生方面，三个协同智能体把一次采样变成一段自主交互经验：执行者在沙箱中以 **ReAct** 方式求解任务，观察者对沙箱前后状态作差得到确定性状态差分，提问者基于上轮经验生成下一轮种子。在奖励可信方面，外部冻结裁判依据环境真实状态差分而非智能体自述打分，从取证机制上抑制奖励作弊。此外，系统以**失败案例驱动评估器与基建的自演化**：周期性收集失败案例，由大模型区分其根因在模型还是基建，进而分别修正提示词或将缺失能力补齐为可复用技能，使自进化从任务与策略扩展到评估器与基建。

系统在 Qwen3.5-9B 上端到端验证，完整自进化链路跑通，沙箱状态跨步继承验证通过。结果表明，少量种子查询即可驱动完整强化学习训练，系统除初始种子

外不依赖任何外部数据。

关键词： 自进化强化学习；多智能体合成数据；差分驱动奖励

Abstract

Large language model agents accomplish tasks through multi-step tool use in real environments such as code sandboxes and file systems, where reinforcement learning has become the dominant approach for continuous improvement. However, RL for interactive agents faces a fundamental bottleneck: both training data and trustworthy rewards are difficult to obtain at scale through human effort. Real user data accumulates slowly with narrow coverage; and rewards that rely solely on agent self-reports lead to reward hacking, where the model learns to falsely claim task completion.

This thesis designs and implements a self-evolving RL system for agentic LLMs, whose primary objective is to enable agents to autonomously generate training data, obtain trustworthy rewards, and produce next-step training tasks without human annotation. The system takes self-evolution as its first goal: beyond the initial seed, all subsequent training queries are autonomously generated based on prior execution experience—an observer summarizes the previous step’s best trajectory, a questioner generates persona-consistent new seeds, and sandbox file-system state is inherited across steps so that the agent continues building on prior outputs. An oversample-eliminate-select mechanism further ensures that only the gradient-optimal trajectory subset is used for training at each step, improving self-evolution efficiency.

For autonomous data generation, three collaborative agents turn each sampling into a self-driven interaction: an actor solves tasks via ReAct in a sandbox, an observer computes deterministic state diffs, and a questioner generates next-step seeds from prior experience. For reward trustworthiness, a frozen judge scores based on real environment state diffs rather than agent self-reports, suppressing reward hacking at the evidence level. In addi-

tion, the system drives self-evolution of the evaluator and infrastructure from failure cases: it periodically collects failure cases, uses an LLM to attribute the root cause to either the model or the infrastructure, and accordingly revises prompts or distills missing capabilities into reusable skills, extending self-evolution from tasks and policy to the evaluator and infrastructure.

The system is validated end-to-end on Qwen3.5-9B, with the full self-evolution pipeline operational and sandbox state inheritance verified. Results show that a small number of seed queries can drive complete RL training, and the system operates without external data beyond the initial seed.

Key Words : Self-Evolving Reinforcement Learning; Multi-Agent Synthetic Data; Diff-Driven Reward

目录

第 1 章 绪论	1
1.1 研究背景	1
1.2 研究问题	3
1.3 研究内容	3
1.4 主要贡献	4
1.5 论文组织结构	5
第 2 章 相关理论与研究现状	7
2.1 大语言模型智能体	7
2.2 智能体评测环境	8
2.3 智能体强化学习与 GRPO	8
2.4 自进化强化学习	9
2.5 奖励建模与奖励作弊	10
2.6 失败归因与评估器、基建的自演化	10
2.7 智能体强化学习系统与训练框架	11
2.8 现有研究分析	12
2.9 本章小结	12

第 3 章 问题建模	15
3.1 研究目标	15
3.2 自进化强化学习过程定义	16
3.3 系统状态与数据流	17
3.4 核心问题	19
3.5 系统边界与基本假设	20
3.6 本章小结	21
第 4 章 自进化强化学习方法设计	23
4.1 方法总体框架	23
4.2 跨步自进化任务生成机制	25
4.3 多智能体协作数据生成机制	26
4.4 超采样-淘汰-选组机制	28
4.4.1 为何训练只需 N 组：理论分析	29
4.5 差分驱动奖励机制	31
4.6 失败案例驱动的评估器与基建自演化机制	32
4.7 自进化稳定性控制机制	34
4.8 本章小结	35
第 5 章 自进化强化学习系统设计与实现	37
5.1 系统总体架构	37
5.2 沙箱环境与状态管理	39
5.3 推理与训练异步调度	40
5.4 跨步状态继承	41

5.5 采样与训练解耦.	42
5.6 自进化数据处理流程	43
5.7 训练监控与自动终止	44
5.8 系统运行流程.	45
5.9 本章小结	48
第 6 章 实验与结果分析	49
6.1 实验环境与参数设置	49
6.2 实验问题与评价指标	50
6.3 自进化闭环验证.	50
6.4 自主数据生成效果	52
6.5 差分驱动奖励效果	53
6.6 超采样-淘汰-选组效果	54
6.7 强化学习训练效果	55
6.8 系统稳定性与效率	56
6.9 失败案例自演化效果	57
6.10 模型能力评测	57
6.11 实验结果讨论	59
6.12 本章小结.	59
第 7 章 总结与展望	61
7.1 工作总结	61
7.2 主要贡献	61
7.3 局限性	62
7.4 研究展望	63

参考文献	65
附录 A 奖励裁判打分细则	71
A.1 系统提示	71
A.2 四维打分细则	71
A.3 交叉验证	74
致谢	75

图目录

图 3.1 每步同步更新的三类状态在训练轮次间的传递：策略、沙箱、任务 . . .	18
图 3.2 评估器与基建的慢节奏演进：失败案例累积触发归因，按模型/基建 根因分路演进版本 $\Phi_{t'}$	19
图 4.1 自进化强化学习方法总体框架	24
图 4.2 跨步自进化闭环	26
图 4.3 多智能体协作数据生成：执行者产轨迹、观察者取证、提问者追问 . .	27
图 4.4 超采样-淘汰-选组机制	28
图 4.5 差分驱动奖励（抗奖励作弊）	31
图 4.6 失败案例驱动的评估器与基建自演化流程	32
图 4.7 自进化稳定性控制：五项监控指标与自动终止判据	34
图 5.1 系统总体架构	38
图 5.2 沙箱环境与状态管理：执行前后快照作差得到确定性状态差分	39
图 5.3 推理与训练异步调度时序：两阶段交替独占 GPU，权重更新后同步 回推理引擎	40
图 5.4 沙箱状态继承机制	41
图 5.5 采样与训练解耦：超采样层 → 选组层 → 训练批次，漏斗式收敛 . . .	42
图 5.6 单训练步内的自进化数据处理流程	43
图 5.7 训练指标与终止判据（奖励均值、奖励标准差、pg_loss、ppo_kl） . .	45
图 5.8 三智能体协作时序	47

图 5.9 执行者沙箱内 ReAct 循环	47
图 6.1 超采样量 S 随训练步收缩	51
图 6.2 消融：去跨步状态继承，与完整系统三指标对比	52
图 6.3 任务难度随自进化演化.	53
图 6.4 消融：去差分驱动奖励，与完整系统三指标对比	54
图 6.5 消融：去超采样-淘汰-选组，与完整系统三指标对比	54
图 6.6 自产生数据（本文）与回流数据（对照）训练的奖励对比（EMA 平滑，阴影为组内奖励标准差）.	56
图 6.7 消融：去失败案例自演化，与完整系统三指标对比	57
图 6.8 三个模型在 coding 任务上的评测对比：平均得分（左）与生成多样性（右）	59

表目录

表 6.1 问题—方法—实验对应关系 49

表 6.2 实验配置 50

第 1 章 绪论

1.1 研究背景

近年来，随着大规模预训练与后训练技术的持续发展，大语言模型（Large Language Model, LLM）在语言理解、文本生成、复杂推理和代码生成等任务中展现出较强的通用能力。与此同时，大语言模型的应用形态逐渐由面向单轮文本生成的模型向能够进行任务规划、调用外部工具并与环境持续交互的智能体（Agent）演进^[1, 2, 3]。与传统文本生成任务相比，智能体通常需要在动态环境中执行由多个步骤组成的复杂任务，其最终表现不仅取决于单次输出的正确性，还与任务规划、工具调用、环境状态变化以及多轮决策过程密切相关。因此，如何进一步提升大语言模型在复杂环境中的多步决策与任务完成能力，成为智能体研究中的重要问题。

强化学习（Reinforcement Learning, RL）通过智能体与环境之间的持续交互，根据任务执行过程中获得的奖励信号优化模型策略，为提升智能体的决策能力提供了重要技术路径。近年来，强化学习逐渐应用于大语言模型的复杂推理、代码执行和智能体任务训练^[4, 5, 6]。其中，以群组相对优势为核心的 Group Relative Policy Optimization (GRPO)^[7]通过比较同一任务下多个采样结果的相对表现构造策略优化信号，在无需显式价值模型的情况下实现模型策略更新，为大语言模型强化学习提供了有效的训练范式。然而，将强化学习应用于智能体场景后，训练数据和奖励评估面临的问题更加突出：一方面，智能体强化学习通常需要大量具有一定复杂度和多样性的任务，而人工构造和标注此类任务的成本较高；另一方面，智能体任务的完成情况往往难以通过单一标准答案进行判断，需要综合工具调用记录、执行轨迹以及外部环境状态进行评估，仅依赖生成轨迹或智能体自反馈可能导致奖励评估偏差。

随着大语言模型能力和应用场景不断扩展，传统依赖人工构造任务和固定数据集的训练范式面临进一步的扩展瓶颈。一方面，固定训练数据难以持续覆盖不断增长的任务空间；另一方面，智能体强化学习过程中需要持续产生新的交互轨迹和训练经验，使训练数据的获取成为限制训练规模和持续迭代能力的重要因素。在此背景下，大语言模型自进化（Self-Evolution）逐渐受到关注^[8,9]。自进化的基本思想是使模型在较少人工干预的情况下，通过“生成—执行—评估—反馈—更新”的循环不断产生新的经验并优化自身能力。其中，自主产生训练数据是实现自进化的重要环节^[10, 11, 12]：模型可以利用已有任务和交互经验生成后续任务，并结合执行结果筛选具有训练价值的经验用于策略优化，从而降低对固定任务集合和持续人工数据采集的依赖。对于智能体强化学习而言，进一步利用环境状态及其变化，可以使任务生成、执行反馈和策略更新形成更加紧密的闭环，使训练过程由固定数据集驱动逐渐转变为由模型、任务和环境共同驱动的动态训练过程。

然而，当任务生成、奖励评估和策略更新均由系统自动完成时，训练闭环同时面临新的风险。如果奖励评估存在偏差，或者生成的数据质量不足，该偏差可能随着后续任务生成和策略更新被进一步放大^[13, 14]。例如，智能体可能通过生成符合评价器偏好的行为获得较高奖励，但实际环境状态并未达到任务要求；低质量或重复任务也可能在多轮任务生成过程中不断出现，使训练数据逐渐偏离预期目标。因此，自进化强化学习系统不仅需要具备持续产生训练数据的能力，还需要建立与环境执行结果相结合的奖励评估机制，并对候选训练经验进行质量控制，从而提高训练闭环的稳定性和有效性。

基于上述背景，本文围绕面向智能体的大模型自进化强化学习系统展开研究。本文以少量人工提供的初始任务作为种子，使智能体在沙箱环境中执行任务并获得交互轨迹与环境状态变化，在此基础上自主生成后续训练任务；同时，通过环境状态差异辅助奖励评估，并结合候选轨迹的超采样、质量淘汰与训练组选择，对进入策略优化阶段的训练经验进行筛选，最终利用强化学习更新模型策略，并将更新后的模型用于下一轮任务生成与执行。通过上述机制，构建从任务生成、环境交互、状态观测、奖励评估、经验筛选到策略更新的持续迭代闭环，探索减少人工任务构造依赖并利用环境交互信息持续产生训练数据的实现方法。

1.2 研究问题

基于上述研究背景，本文关注的核心问题是：

能否构建一种面向智能体的自进化强化学习系统，以少量人工提供的初始任务作为种子，使模型在训练过程中自主产生后续训练任务与交互经验，并利用环境反馈构建奖励信号，通过有效的数据筛选与策略优化形成能够持续运行的自进化训练闭环？

围绕这一核心问题，本文进一步将研究内容分解为以下四个子问题：

子问题一（训练数据自主产生）：如何降低智能体强化学习对人工构造任务和外部真实数据的依赖，使系统能够利用已有任务与交互经验自主产生具有训练价值的后续任务？

子问题二（环境反馈与奖励评估）：如何利用交互过程中产生的客观环境状态变化辅助奖励评估，降低仅依赖智能体自述所带来的奖励误判，提高训练信号与实际任务完成情况的一致性？

子问题三（评估器与基建自进化）：在任务与策略之外，如何使评估器与执行基建也随训练持续改进——即根据失败案例在模型侧修正评估标准、在 harness 侧补齐缺失的环境能力，从而缓解奖励评估与执行环境的固有缺陷对训练闭环的制约？

子问题四（自进化闭环构建）：如何将任务生成、环境交互、奖励评估、经验筛选和策略更新组织为持续迭代的训练流程，使上一轮产生的经验驱动下一轮任务生成，并支持系统在预设终止条件下自主运行与监控？

1.3 研究内容

针对上述研究问题，本文设计并实现一套面向智能体的大模型自进化强化学习系统，主要研究内容包括：

（1）**自进化强化学习问题建模。**将传统“固定任务集—轨迹采样—奖励计算—策略更新”的强化学习训练流程扩展为“任务生成—环境交互—状态观测—奖励评估—经验筛选—策略更新—任务再生成”的闭环过程，并对系统中的任务状态、

环境状态和策略状态进行形式化描述，为后续方法设计与系统实现提供统一的问题模型。

(2) **自进化强化学习方法设计**。针对训练任务自主产生、交互经验质量控制以及训练反馈可靠性等问题，设计跨步自进化任务生成、多角色协作数据生成、超采样—淘汰—选组、差分驱动奖励、失败案例驱动的评估器与基建自演化以及自进化稳定性控制六个机制，使任务生成与模型训练能够在统一闭环中持续迭代。

(3) **自进化强化学习系统实现**。面向上述方法设计实现完整训练系统，包括沙箱环境、跨步状态继承、推理与训练异步调度、采样与训练解耦、数据与奖励处理、训练状态监控以及自动终止等工程模块，实现自进化强化学习流程的自动化运行。

(4) **实验验证与分析**。构建基础模型、传统强化学习训练流程及本文完整系统等对照实验，并针对任务生成、状态继承、奖励评估、候选数据筛选等关键机制开展消融实验，从任务生成质量、训练过程稳定性和独立测试任务上的模型能力等多个维度分析系统的有效性与适用范围。

1.4 主要贡献

本文围绕面向智能体的大模型自进化强化学习训练过程，主要贡献如下：

贡献一：提出跨步自进化任务生成机制。在单轮任务生成的基础上，将上一轮智能体执行产生的任务结果、环境状态及相关执行信息用于后续任务生成，使系统能够从少量初始种子出发持续产生后续训练任务，形成跨训练步骤的任务演化过程。

贡献二：构建多角色协作的自主数据生成流程。将任务执行、环境状态观测和后续任务生成划分为不同功能角色，通过执行者、观察者和提问者之间的协作，实现训练任务、交互轨迹和环境反馈的自动产生，减少人工介入训练数据生成过程的需求。

贡献三：设计超采样—淘汰—选组的训练数据筛选机制。针对智能体强化学习中候选交互轨迹数量较多而有效训练样本有限的问题，通过增加候选轨迹数量，并依次进行低质量样本淘汰和训练组选择，在控制实际训练数据规模的同时提高候选训练经验的利用效率，实现“多推理、少训练”的数据筛选流程。

贡献四：设计差分驱动的奖励评估机制。针对仅依据智能体自述或生成轨迹进行奖励评估可能产生偏差的问题，引入沙箱环境状态差异作为奖励评估的辅助证据，将智能体行为与实际环境变化进行关联，以降低奖励评估对智能体自反馈的依赖，提高奖励信号的环境可验证性。

贡献五：提出失败案例驱动的评估器与基建自演化机制。在任务与策略之外，将评估器与基建也纳入自进化对象：训练过程中周期性收集失败案例，由大模型综合任务意图、执行轨迹与环境证据判断失败根因在模型还是基建，进而分别对评估提示词进行增量修订或将缺失的基建能力总结为可复用的技能予以补齐，使评估与执行环境随训练持续自我完善。

贡献六：提出自进化稳定性控制机制。针对任务生成、奖励评估与策略更新均由系统自动完成时训练闭环可能出现的偏差累积与状态漂移问题，围绕候选数据规模控制、奖励信号约束以及训练状态监控与自动终止等方面设计稳定性控制策略，抑制低质量任务和奖励偏差在多轮迭代中的放大，保障自进化训练过程在预设条件下的持续稳定运行。

1.5 论文组织结构

根据本文的研究内容和研究思路，全文共七章，各章节主要内容如下：

- 第一章为绪论，介绍本文的研究背景、研究问题、研究内容、主要贡献以及论文整体组织结构。
- 第二章为相关理论与研究现状，介绍大语言模型智能体、智能体评测环境、智能体强化学习与 GRPO、自进化学习、奖励建模与奖励作弊以及相关训练框架，并对现有研究进行分析。
- 第三章建立面向智能体的大模型自进化强化学习问题模型，对智能体与环境的交互过程、自进化过程中的系统状态、训练数据与奖励信号进行形式化描述，并明确本文研究的问题边界。
- 第四章为自进化强化学习方法设计，详细介绍跨步自进化任务生成、多角色

协作数据生成、超采样—淘汰—选组、差分驱动奖励、失败案例驱动的评估器与基建自演化以及自进化稳定性控制六个核心机制。

- 第五章为系统设计与实现，介绍系统总体架构、沙箱环境、状态继承、异步调度、采样与训练解耦、训练数据处理以及训练监控等模块，并给出系统整体运行流程。
- 第六章为实验与结果分析，通过基础模型与强化学习基线实验、关键机制消融实验以及完整系统实验，分析本文方法在任务自主生成、奖励评估、训练数据筛选、训练稳定性和模型能力提升等方面的表现。
- 第七章为总结与展望，总结本文的主要研究工作和实验结论，分析当前工作的局限性，并对未来可能的研究方向进行展望。

第 2 章 相关理论与研究现状

本章综述与本文直接相关的理论基础与研究现状。其中 2.1–2.2 介绍大语言模型智能体及其评测环境，作为全章的共性背景；自 2.3 起各节大体沿本文方法的脉络组织：智能体强化学习与 GRPO (2.3–2.4)、自进化强化学习 (2.5)、奖励建模与奖励作弊 (2.6)、失败归因与评估器/基建自演化 (2.7)，最后介绍智能体强化学习系统与训练框架 (2.8)。在此基础上分析现有研究的不足，明确本文的定位。

2.1 大语言模型智能体

大语言模型智能体在代码沙箱、终端、文件系统等真实环境中，通过“思考—调用工具—观察结果”的循环完成多步任务。**ReAct**^[1] 首先将推理与行动统一在语言模型的生成过程中，使模型能够以交错的思维链与工具调用求解多步问题，成为当前交互式智能体的基础范式。**Toolformer**^[2] 证明语言模型可以通过自监督方式学习何时、如何调用外部工具，将 API 调用内化为模型能力。在交互经验的组织层面，**Reflexion**^[15] 让智能体以语言化的自我反思积累经验并在后续尝试中改进；**Voyager**^[16] 在开放游戏环境中以自动课程驱动技能库的持续扩张，是“经验自主积累”的早期代表。**CodeAct**^[3] 进一步表明，以可执行代码作为统一动作空间比离散工具调用更精确、更可组合，本文的执行者即采用代码沙箱内的 **ReAct** 循环。多智能体协作方面，**CAMEL**^[17] 通过角色扮演的双智能体会话规模化地生成指令数据，验证了“以智能体对话产生训练数据”的可行性，这一思路与本文的提问者—执行者协作直接相通。

这类交互式智能体的能力提升，既依赖模型本身的推理能力，也依赖在真实环境中的大量交互经验。如何低成本地获得这些交互经验，是智能体训练的关键问

题。

2.2 智能体评测环境

智能体研究的发展伴随着评测环境的成熟。WebShop^[18] 构建了首个大规模真实网页购物环境,支持以自然语言指令驱动的多步网页交互;WebArena^[19] 将其扩展为可自托管的现实网站集合,成为网页智能体训练与评测的标准环境。在软件工程方向,SWE-bench^[20] 从真实 GitHub issue 构建任务,要求模型在真实仓库中定位并修复缺陷;SWE-agent^[21] 设计了智能体—计算机接口(ACI),显著提升了智能体在该类环境中的表现。通用评测方面,AgentBench^[22] 汇聚八个环境,系统评测大模型作为智能体的多步决策能力;AgentBoard^[23] 提供多轮智能体的细粒度分析框架。操作系统层面,OSWorld^[24] 提供真实计算机环境中的开放任务基准;TheAgentCompany^[25] 则模拟真实公司组织中的长程任务。这些环境覆盖了网页、代码、终端、桌面等多种交互场景,但它们主要作为**静态评测基准**存在:任务集合固定、人工构建,训练阶段难以直接从中持续获得与策略能力匹配的新任务——这正是本文自进化任务生成所要弥补的缺口。

2.3 智能体强化学习与 GRPO

强化学习通过交互轨迹上的奖励信号直接优化策略,是提升交互式智能体能力的主流手段。策略优化层面,PPO^[26] 以裁剪目标函数实现稳定的策略梯度更新,是 RLHF 时代的默认算法;DPO^[27] 则绕过显式奖励模型直接从偏好数据优化策略,但难以处理带环境反馈的序列决策。GRPO (Group Relative Policy Optimization)^[7] 对同一查询的一组 rollout 做组内奖励归一化来估计优势,省去了价值网络,成为当前智能体强化学习的主力算法。围绕 GRPO 的分析持续深入:Dr.GRPO^[28] 指出其长度归一化引入的长度偏置;DAPO^[29] 通过解耦裁剪与动态采样在大规模训练中提升稳定性;理论工作^[30] 进一步揭示 GRPO 的方差缩减主要由每步 rollout 总数(查询数 × 组大小)驱动,为通过扩大采样规模提升训练信号质量提供了理论依据——本文的超采样-淘汰-选组机制正是对这一结论的系统性利用。在多轮智能体训

练中，ArCHer^[31] 提出分层 actor-critic 分离轮级与 token 级信用分配；GiGPO^[32] 以组内分组结构降低多轮优势估计的方差；RAGEN^[33] 则系统研究了多轮训练中的策略多样性坍塌（Echo Trap）现象：策略在训练中逐渐丧失多样性、所有轨迹趋于一致，导致梯度信号消失、训练停滞。这一发现直接支撑了本文将稳定性监控作为自进化系统的核心组件。

在真实环境中的智能体强化学习已有一批标志性实践：UI-TARS^[34] 在 GUI 环境中以大规模合成轨迹与在线 RL 训练端到端 GUI 智能体；GLM-4.5^[35] 将智能体能力作为核心训练目标；Kimi k1.5^[36] 与 DeepSeek-R1^[4] 证明了大规模强化学习能够有效激发推理能力；Kimi K2^[37] 与通义 DeepResearch^[38] 将 agentic 强化学习推进到工具调用与深度研究场景；Search-R1^[5] 训练模型在推理中主动调用搜索引擎；SWE-RL^[39] 在软件演化数据上进行基于规则奖励的强化学习；SWEET-RL^[40] 研究多轮协作任务的信用分配；AgentGym-RL^[6] 在多环境中进行多轮强化学习以训练长程决策。这些工作验证了智能体强化学习的有效性，但它们的任务供给依赖固定数据集或人工构建的环境内置信号，奖励依赖可验证规则或环境回报，尚未解决开放交互场景下数据与奖励的自主供给问题。

2.4 自进化强化学习

自进化的核心思想是让模型通过自身产生的经验持续优化，而非依赖固定的外部数据集。在文本推理层面，这一方向已有系统性积累：Self-Improve^[9] 以自一致性筛选自生成答案实现自我提升；Self-Instruct^[10] 与 Evol-Instruct^[41] 以自生成与演化指令扩充训练数据；ReST^[42] 迭代式地在自采样数据上训练；SPIN^[43] 将自我提升形式化为自我博弈。更进一步，Self-Rewarding^[44] 让模型自评偏好充当奖励信号；SEAL^[45] 让模型自行生成自我改写以更新自身；Absolute Zero^[11] 以“提案—求解”双角色自我博弈，在零外部数据下训练推理器；R-Zero^[12] 用导师—求解者—验证者三角角色从零数据自进化出推理模型，其多角色分工与本文的三智能体协作在结构上相近。RFT 的持续微调研究^[46] 还表明，强化学习式微调比监督微调更易遗忘，为以强化学习驱动持续进化提供了动机。

在交互式智能体层面，自进化也开始被探索：AgentGym^[47] 让智能体在多环境

中以指令进化持续提升能力；WebRL^[48] 为网页智能体构建自进化在线课程——从失败任务合成新指令并动态调整难度，是“任务自主生成驱动智能体训练”的代表性工作。自进化方向的综述^[8] 将这类系统归纳为“经验积累—自我反思—能力更新”的循环。总体而言，现有自进化工作在核心思想上与本文一致——基于历史经验自主产生新训练信号，但它们主要停留在**文本层面**：任务由语言模板演化生成，环境反馈以文本匹配或规则判分给出，未涉及交互式智能体的**环境状态**在训练轮次间的传递与继承。而后者恰是真实沙箱环境中自进化的关键——智能体在上轮产出的文件与配置上继续工作，任务与状态才能形成有意义的演化链。

2.5 奖励建模与奖励作弊

奖励的可信性是强化学习的核心问题。RLHF^[49] 以人工偏好训练奖励模型，但奖励模型本身可能被策略过度优化——Goodhart 效应使得策略学会欺骗代理奖励而非真正完成任务^[50, 14]。Skalse 等^[13] 对奖励作弊 (reward hacking) 给出了形式化定义与刻画，指出其根源在于代理奖励与真实目标的对齐偏差。可验证方向上，GSM8K 的验证器^[51] 与过程奖励模型 PRM^[52] 以最终答案或逐步判分提供确定性信号，但仅适用于答案可验证的封闭任务。RLAIF^[53] 与 LLM-as-a-judge^[54] 将裁判推广到开放任务，生成式验证器^[55] 进一步将奖励建模统一为下一个 token 预测。然而近期研究警示了奖励信号的脆弱性：Spurious Rewards^[56] 发现在部分基座上随机奖励也能产生与真实奖励相当的提升，说明奖励信号与能力提升之间的因果关系远比预期复杂。对交互式智能体而言，问题更加尖锐：若裁判看到的只是智能体自述的轨迹文本，模型会迅速学会在轨迹中谎报完成，而裁判无从核实。如何让奖励锚定**环境真实状态**而非智能体叙事，是奖励可信化的核心问题，当前尚未被充分解决——本文以确定性状态差分为取证依据的奖励机制正是针对这一问题。

2.6 失败归因与评估器、基建的自演化

除任务与策略外，评估器（打分与提问的提示词）与工具集/基建也是可被自主演化的对象，这一方向的研究可归纳为三条线索。其一，**失败归因与自我调试**：Re-

flexion^[15] 让智能体以语言化反思定位失败原因并在后续尝试中修正；Self-Debug^[57] 教会大模型通过阅读执行结果解释并调试自身生成的代码，说明大模型具备对失败案例做根因分析的能力。其二，**提示词自动优化**：APE^[58] 将提示词搜索形式化为大模型驱动指令生成与筛选，达到人类水平的提示词工程；ProTeGi^[59] 以自然语言“梯度”与束搜索迭代改写提示词，验证了提示词可依据失败样本被自动、增量地修订。其三，**工具与技能的自主创建**：Voyager^[16] 在开放环境中将成功经验固化为可复用的技能库并持续扩充；CREATOR^[60] 让大模型在缺乏现成工具时自行创建工具以解决问题；Toolformer^[2] 则表明工具使用能力可被模型自监督地内化。这些工作分别证明了失败可被大模型归因、提示词可被自动优化、工具/技能可被自主创建。

然而，现有工作大多在**推理或部署阶段**、针对单个任务孤立地进行反思、调试或工具创建，尚未将“失败归因—提示词/基建演化”整合进**强化学习训练闭环**，也很少区分失败的根因究竟在**模型**（策略的规划、工具使用、方案选择失误）还是在**基建**（沙箱、工具、执行环境的缺陷）——而这一区分恰恰决定了应当修正提示词还是扩展基建。本文正是针对这一空白，在训练过程中周期性收集失败案例，由大模型综合任务意图、轨迹与环境证据做出模型/基建的归因，进而分别演化评估器提示词或将缺失的基建能力总结为可复用的技能（skill）予以补齐，使评估器与基建随训练持续自我完善。

2.7 智能体强化学习系统与训练框架

大规模强化学习系统为智能体训练提供工程底座。推理侧，vLLM^[61] 以 Page-dAttention 实现高吞吐内存管理，成为主流推理引擎。训练框架侧，verl (HybridFlow)^[62] 以混合编程模型统一单控制器的灵活性与工作流的高效性，支持推理与训练的灵活编排，本文即以其官方注入点接入自定义采样与选组层；OpenRLHF^[63] 提供易用的全栈 RLHF 管线；AReaL^[64] 以大规模异步架构解耦 rollout 与训练，提升异构任务的吞吐；Agent Lightning^[65] 专注任意交互式智能体的 RL 微调，提供无需修改智能体实现的训练接入。这些框架解决了“如何高效训练”的问题，但框架本身不提供数据自主产生、奖励可信取证、跨步状态继承等自进化所需的核心机制，这些

机制需在框架之上自行设计与集成。

2.8 现有研究分析

综合上述方向，现有研究呈现如下格局：算法层面（PPO、GRPO 及其变体）已较成熟，大规模智能体强化学习实践已有充分验证；评测环境覆盖网页、代码、终端等场景，但作为静态基准存在；自进化方向已有文本层面的系统探索（Self-Instruct、SPIN、Absolute Zero、R-Zero），并开始进入智能体场景（AgentGym、WebRL）；奖励建模已有 RLHF/RLAIF/裁判方案与作弊理论刻画；工程层面已有成熟的训练框架。然而，将这些要素整合为一套“自主、可信地供给训练数据与奖励，并持续自进化运转”的完整系统，仍存在明显空白：

（1）数据供给仍依赖人工采集或固定数据集，缺乏交互式智能体在真实沙箱环境中的自主数据产生机制；

（2）奖励仍主要基于智能体自述文本或可验证规则，缺乏锚定环境真实状态的取证机制，难以在开放任务上同时兼顾覆盖面与抗作弊性；

（3）自进化仍停留在文本层面，任务由模板演化生成、反馈以规则判分，缺乏交互环境状态在训练轮次间的继承，任务难度与环境实际状态脱节。

（4）失败归因与提示词/工具自演化虽已有研究，但多在部署阶段针对单任务孤立进行，未整合进训练闭环，也很少区分失败根因在模型还是基建，缺乏据此分别演化评估器提示词或补齐基建能力的机制。

本文正是面向这一空白，以多智能体协作合成数据、差分驱动奖励、跨步自进化闭环（含沙箱状态继承与稳定性监控）、以及失败案例驱动的评估器与基建自演化，构建一套自进化强化学习系统。

2.9 本章小结

本章综述了大语言模型智能体与评测环境、智能体强化学习与 GRPO、自进化学习、奖励建模与奖励作弊、训练框架等方向的理论基础与研究现状，并分析指出：现有工作在算法与工程层面已较成熟，但在“如何自主、可信地供给训练数据与奖

励，并使系统在真实环境中持续自进化运转”这一系统与数据层面仍有明显空白。本文的工作正是面向这一空白展开。

第 3 章 问题建模

本章建立面向智能体的自进化强化学习问题模型：先给出研究目标，再形式化描述自进化过程，明确系统状态与数据流，最后界定核心问题与系统边界。

3.1 研究目标

本文的研究目标是构建一套自进化强化学习系统，使交互式智能体在无人工标注条件下满足以下四个目标：

目标一：数据自主产生。训练数据的获取基于交互生成——模型在与环境的交互（沙箱内工具调用）以及智能体间的协作（提问者追问、观察者取证）中动态产生新的经验与数据，而非依赖真实用户回流或人工采集。除少量初始种子用于冷启动外，后续训练数据由系统在交互过程中持续自主合成。

目标二：奖励可信。奖励锚定在环境真实状态而非智能体自述，从取证机制上抑制奖励作弊。

目标三：自进化闭环。系统形成“采样—奖励—更新—生成下一轮任务”的闭环，持续自主运转，无需人工干预。

目标四：评估器与基建自演化。系统不仅演化任务与策略，还应能根据训练中暴露的失败案例，自主演化评估器（提示词）与基建（工具/沙箱能力）：区分每一类的失败案例是来源于评估器缺陷导致的模型评分错误、梯度信号错误，还是训练基建，如沙箱缺陷、常用 Agent 工具缺失等，据此分别修正对应的提示词、算法或修复基建缺陷，将缺失能力补齐为可复用的技能，使评估与执行环境随训练持续完善，以此提供更加纯粹的模型更新信号和更加完善的 Agent 运行环境，确保 Agent 自进化的顺利推进。

3.2 自进化强化学习过程定义

传统强化学习在固定任务集上采样、打分、更新策略，任务集由外部供给且训练过程中保持不变。本文将其扩展为一个自进化过程：任务本身由系统基于上一轮经验动态生成。

设第 t 步的任务集为 Q_t ，被训练的策略为 π_{θ_t} ，当前生效的评估器与基建状态为 $\Phi_{t'}$ 。一个训练步的过程可形式化描述为：

$$Q_t \xrightarrow{\pi_{\theta_t}} \tau_t \xrightarrow{\text{Observer}} d_t \xrightarrow{\text{Judge}(\cdot; \Phi_{t'})} R_t \xrightarrow{\text{GRPO}} \theta_{t+1} \xrightarrow{\text{Questioner}} Q_{t+1} \quad (3.1)$$

这里使用两个解耦的下标： t 是**训练步**计数，每个训练步递增一次； t' 是**评估器与基建的演化版本**计数。二者节奏不同——策略 θ 与任务 Q 每个训练步都更新（下标 t ），而评估器与基建**并非每轮都更新**：系统在训练中持续收集失败案例集 $\mathcal{B}_{t'}$ ，仅当满足触发条件（每隔若干训练步或攒够一定数量失败案例）时，才由大模型对其归因并演化一次评估器与基建状态，使版本号 t' 递增一次：

$$\mathcal{B}_{t'} \xrightarrow{\text{Attribution}} \Phi_{t'+1} \quad (3.2)$$

因此在任一训练步 t ，Judge 使用的都是**当前最新版本** $\Phi_{t'}$ ； t' 随训练推进而缓慢增长，满足 $t' \leq t$ 。

其中各符号含义为：

- Q_t ：第 t 步的任务集。仅初始任务集 Q_0 来自少量人工种子，后续 Q_t ($t \geq 1$) 均由系统在跨步时生成。
- τ_t ：策略 π_{θ_t} 在沙箱环境中对 Q_t 执行产生的交互轨迹集。
- d_t ：观察者对沙箱执行前后状态作差得到的确定性状态差分，作为奖励的客观依据。
- R_t ：外部冻结裁判在当前评估器与基建状态 $\Phi_{t'}$ 下，依据状态差分 d_t 给出的奖励。
- θ_{t+1} ：经 GRPO 更新后的策略参数。

- Q_{t+1} : 提问者基于第 t 步的执行经验与沙箱状态生成的下一轮任务集。
- $\Phi_{t'}$: 第 t' 个演化版本的评估器与基建状态, 包含各智能体(裁判、提问者、执行者)的提示词与工具/沙箱等基建能力。 Φ_0 为人工设定的初始配置, 后续由失败案例驱动演化; 下标 t' 与训练步 t 解耦, 仅在触发一次演化时递增。
- $\mathcal{B}_{t'}$: 第 t' 个演化周期内累积的失败案例集(执行失败、奖励异常或自述与状态差分矛盾的轨迹及其取证信息)。
- $\Phi_{t'+1}$: 大模型对 $\mathcal{B}_{t'}$ 归因后, 据根因(模型侧或基建侧)分别修订提示词或补齐基建能力而得到的新一版评估器与基建状态。

该过程的关键在于其自主演化的两条支路: 一是 $\theta_{t+1} \xrightarrow{\text{Questioner}} Q_{t+1}$, 任务与策略在训练轮次间协同演化, 而非固定不变; 二是 $\mathcal{B}_{t'} \xrightarrow{\text{Attribution}} \Phi_{t'+1}$, 评估器与基建也随失败案例持续自我完善。二者共同将自进化的对象从传统强化学习仅有的“策略”扩展到“任务、策略、评估器、基建”四维, 这是本文与传统强化学习的本质区别。

3.3 系统状态与数据流

自进化过程涉及四类状态在训练轮次间传递:

(1) **策略状态** θ_t : 被训练的模型参数, 经 GRPO 更新在轮次间演化。

(2) **沙箱状态** s_t : 每个轨迹对应的沙箱文件系统状态。跨步时, 上一轮最优轨迹的沙箱状态继承到下一轮新沙箱, 使智能体在上轮产出的物理基础上继续工作。

(3) **任务状态** Q_t : 任务集。仅初始任务集 Q_0 来自少量人工种子, 后续跨步时由提问者基于执行概括与沙箱状态生成。

(4) **评估器与基建状态** $\Phi_{t'}$: 各智能体的提示词与工具/沙箱等基建能力。初始配置 Φ_0 由人工设定, 后续由失败案例经大模型归因后分路演化(模型侧修订提示词、基建侧补齐能力)。其演化版本号 t' 与训练步 t 解耦: 评估器与基建并非每个训练步都更新, 仅在触发条件满足时演化一次。

四类状态的传递构成自进化的数据流: 策略经奖励更新、沙箱状态经快照继承、任务经提问者再生成、评估器与基建经失败案例归因演化, 四者共同驱动系

统持续演化。其中策略、沙箱、任务三类在每个训练步同步更新，其相邻训练步之间的传递关系如图 3.1 所示；评估器与基建则为解耦的慢节奏演进支路，另行说明（见图 3.2）。

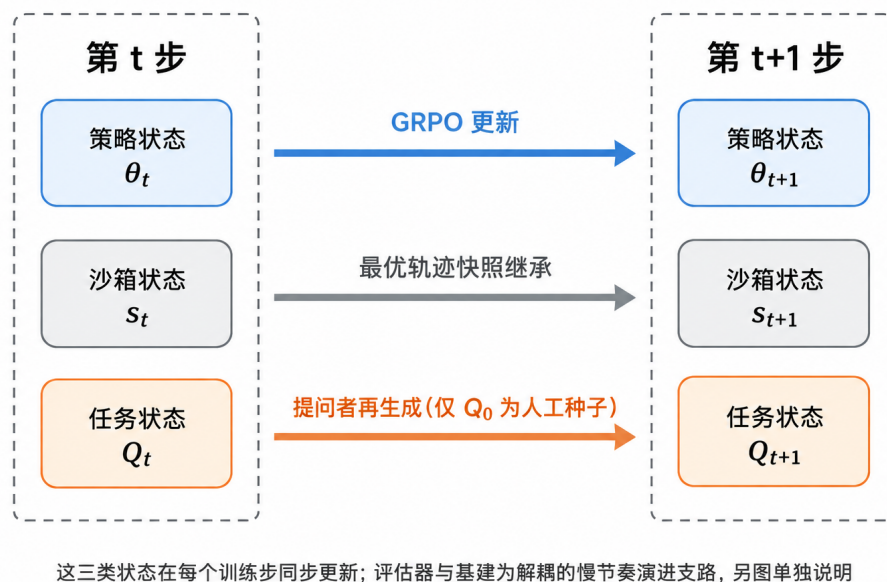


图 3.1 每步同步更新的三类状态在训练轮次间的传递：策略、沙箱、任务

在上述四类状态中，前三类（策略、沙箱、任务）在**每个训练步**同步更新，构成主循环的快节奏；而评估器与基建状态 Φ_{ν} 则遵循一条**解耦的慢节奏演进支路**，需要单独说明。该支路的运行方式为：系统在训练过程中持续将失败案例（执行失败、奖励异常、自述与状态差分矛盾的轨迹）收入缓冲区 $\mathcal{B}_{\nu}$ ；每当满足触发条件（每隔若干训练步、或缓冲区累积到一定数量），即冻结当前批失败案例，交由大模型进行一次**根因归因**——判断每个失败的根因落在**模型侧**（策略的规划、工具使用或方案选择失误）还是**基建侧**（沙箱缺依赖、工具缺失、超时过短、harness 处理缺陷）。归因结果分两路演进：一路指向**评估器/提示词侧**，对评估器提示词进行增量修订（裁判的打分口径、提问者的任务生成、执行者的行为约束）；另一路指向**基建侧**，扩展系统基建（注册新工具、补装依赖、调整超时或并发、修正

harness 处理逻辑，或将缺失能力沉淀为可复用技能)。一次演进使版本号 t' 递增一次，随后清空缓冲区、进入下一轮收集。因此评估器与基建并非静态配置，而是沿 $\Phi_0 \rightarrow \Phi_1 \rightarrow \Phi_2 \rightarrow \dots$ 逐版演进；由于触发是间歇性的， t' 的增长显著慢于训练步 t ($t' \leq t$)。这条演进支路与主循环的时间关系如图 3.2 所示。

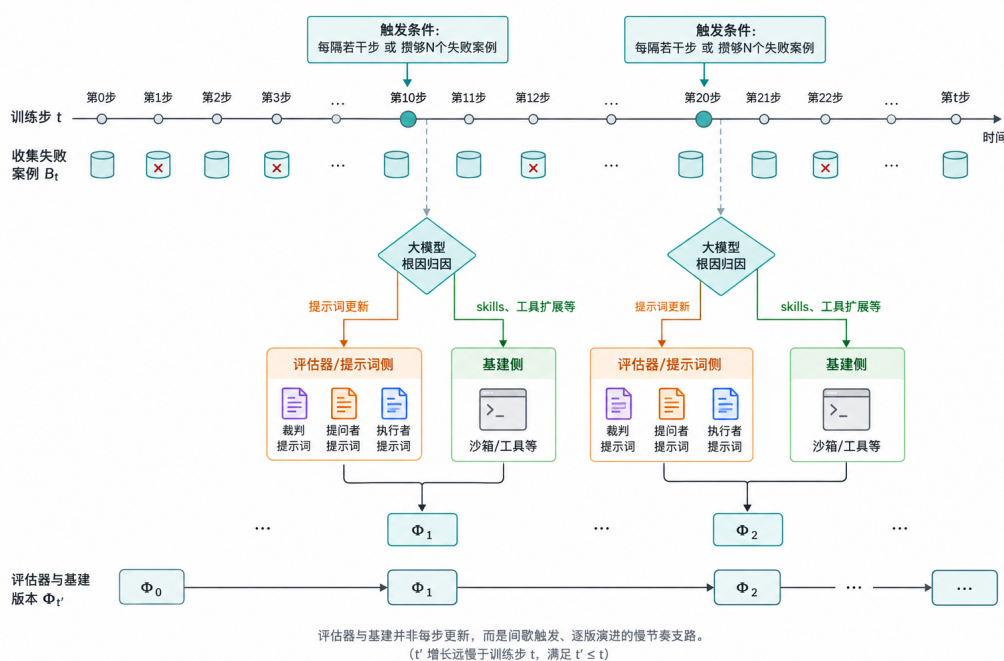


图 3.2 评估器与基建的慢节奏演进：失败案例累积触发归因，按模型/基建根因分路演进版本 $\Phi_{t'}$

3.4 核心问题

围绕上述过程，本文需要解决四个核心问题：

核心问题一：如何自主产生交互轨迹与下一轮任务？这是自进化数据供给的核心。与依赖人工标注数据、人工构建运行环境的传统范式不同，自进化系统需在极少人工介入下自主产生训练数据：一方面借助系统内多智能体的协作（执行者探索、观察者取证、提问者追问）与历史经验持续生成新的交互轨迹与后续任务，另

一方面以上一轮沙箱状态为基础、通过状态的跨步继承与演化产生新的运行环境，从而支撑训练闭环的持续运转。

核心问题二：如何使奖励锚定环境真实状态？人工干预的减少会使训练过程更难约束，也为奖励作弊提供了更大空间——策略可能仅在自述轨迹中谎报完成而裁判无从核实。因此在此场景下抗奖励作弊尤为关键，需要观察者产出确定性的环境状态差分，使裁判据此打分，而非依赖智能体的自我叙述。

核心问题三：如何保证自进化过程的稳定性？自进化系统长时间无人值守地自主运转，一旦出现策略坍缩或训练崩溃而未被及时发现，后续所有训练步都将在无效信号上空转，白白消耗大量计算资源。因此需要实时监控关键指标，并在坍缩或崩溃发生时自动终止训练。

核心问题四：如何让评估器与基建随训练自我完善？训练中会持续暴露系统性失败，需要一种机制自动收集失败案例，并由大模型判别其根因落在**模型侧**（策略的规划、工具使用或方案选择失误）还是**基建侧**（沙箱、工具、执行环境的缺陷），进而分别对评估器提示词进行增量修订、或将缺失的基建能力补齐为可复用技能，避免同类失败反复积累。

3.5 系统边界与基本假设

本文的系统边界与基本假设如下：

- 基座模型为 Qwen3.5-9B，策略优化采用标准 GRPO，不追求算法最优，仅为证明系统可行性。
- 沙箱环境为代码沙箱（本地或云端），支持文件系统操作与命令执行。
- 奖励裁判为外部冻结的大模型，以服务形式提供，训练过程中其**模型权重**保持不变；但其打分所用的**提示词**属于评估器状态 Φ_t ，可由失败案例驱动演化。
- 训练步数不设固定值，由稳定性终止判据决定。

3.6 本章小结

本章建立了自进化强化学习的问题模型：将传统固定任务集的训练范式扩展为任务动态生成的闭环过程，形式化描述了 $Q_t \rightarrow \pi_{\theta_t} \rightarrow \tau_t \rightarrow R_t \rightarrow \theta_{t+1} \rightarrow Q_{t+1}$ 的自进化主循环，以及 $\mathcal{B}_{t'} \rightarrow \Phi_{t'+1}$ 的评估器与基建演化支路，明确了策略、沙箱、任务、评估器与基建四类状态的传递，并界定了四个核心问题与系统边界。后续章节围绕这些问题展开方法设计与系统实现。

第 4 章 自进化强化学习方法设计

根据系统的组织思路和贡献，本章按“总—分”结构展开方法设计：先给出统领全局的方法总体框架，再依次阐述与绪论章所述的六项主要贡献一一对应的六个核心机制。本章聚焦系统设计的原理和运作机制；各机制相应的工程实现步骤见第五章。六个核心机制依次为：跨步自进化任务生成、多智能体协作数据生成、超采样-淘汰-选组、差分驱动奖励、失败案例驱动的评估器与基建自演化、自进化稳定性控制。

4.1 方法总体框架

传统非自进化的智能体强化学习遵循“固定任务集—采样—固定评估函数给予奖励—更新策略”的开环流程：任务由外部一次性给定、评估函数在训练全程固定，模型只是被动地在既有数据上优化策略。本文将扩展为“任务生成—环境交互—状态观测—奖励评估—策略更新—任务再生成”的闭环过程：任务不再外部给定，而是由系统基于上一轮执行经验持续再生成；奖励不再依赖固定评估函数，而是锚定于观察者对环境真实状态的观测；每一轮的策略与产物又成为下一轮任务生成的起点，从而让任务、经验与策略在闭环中协同演化。方法总体框架如图 4.1 所示。

一个训练步内， S 个种子任务经执行者在沙箱中与环境交互并 rollout 产生 $S \times n$ 条轨迹（ n 为 GRPO 组大小），观察者对每条轨迹产出确定性状态差分，裁判据此打分；随后淘汰低质组、选梯度最优组做 GRPO 更新；训练步结束后，系统对存活组产生下一轮种子任务，形成自进化闭环。第一步种子数 $S = 2N$ （ N 为训练组数），后续步 $S = R$ （上一轮存活组数）。

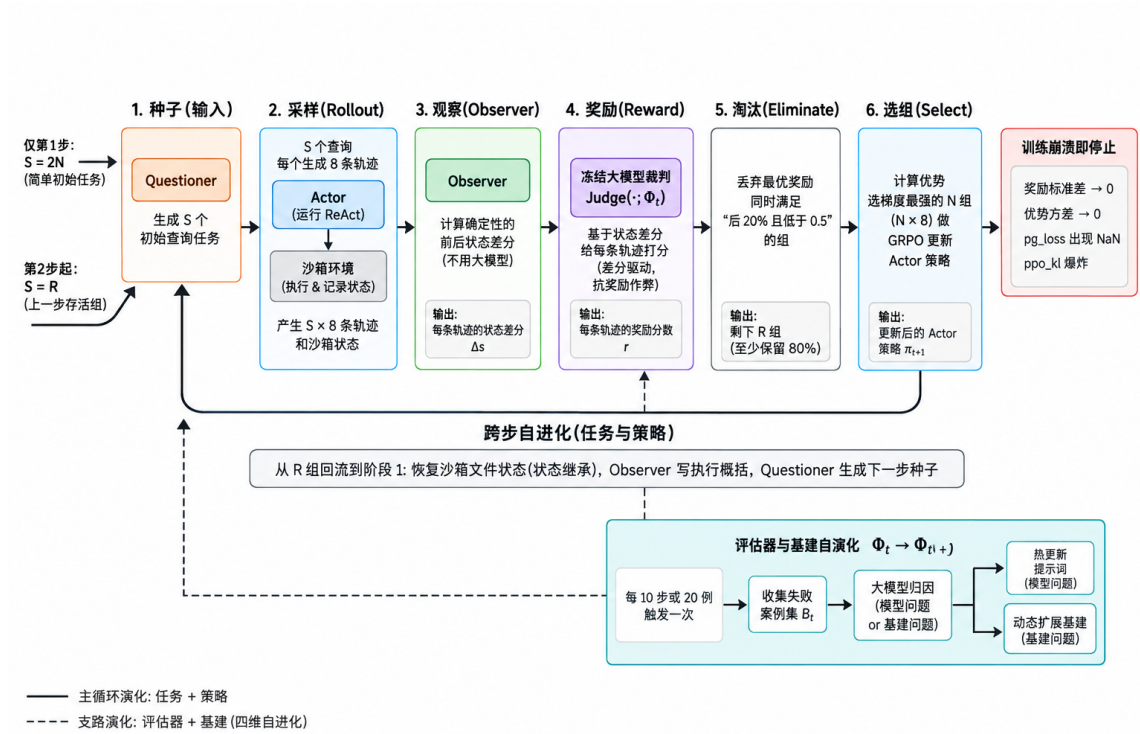


图 4.1 自进化强化学习方法总体框架

上述闭环之上，本文的自进化实际包含**两条节奏不同的演化支路**，与第三章的形式化定义一致。其一是**快节奏主循环**：任务、策略与沙箱状态在每个训练步同步演化——策略经 GRPO 更新、任务由提问者跨步再生成、沙箱状态跨步继承。其二是**慢节奏支路**：评估器与基建状态 Φ_t 并非每步更新，而是由失败案例驱动、周期性触发地演化——系统持续收集失败案例，累积到触发条件（每隔若干训练步或攒够一定数量）时，由大模型归因并分两路更新：评估器/提示词侧（修订裁判、提问者、执行者的提示词）与基建侧（扩展工具、依赖、沙箱能力，或沉淀可复用技能）。这条支路使评估口径与执行环境随训练持续自我完善，其版本号 t' 的增长显著慢于训练步 t 。两条支路共同将自进化的对象从传统强化学习仅有的“策略”扩展到“任务、策略、评估器、基建”四维。

以下六节分别阐述框架中的各个机制。

4.2 跨步自进化任务生成机制

跨步自进化是本文的核心机制，对应**贡献一**，使系统除初始种子外自主产生所有后续训练任务。跨步自进化闭环如图 4.2 所示。

每个训练步完成后，系统执行以下跨步流程：

(1) 取存活组最优轨迹。从淘汰后的存活组中，每组取奖励最高的轨迹及其沙箱状态。

(2) 执行概括。观察者用大模型概括上轮最优轨迹的执行情况——做了什么、产出了什么、还差什么。这一步用概括而非直接复述轨迹，避免沿用原始轨迹文本导致模型偏移。

(3) 沙箱状态继承。将上轮最优轨迹的沙箱文件系统状态传递到本轮新沙箱，使智能体在上轮产出的文件基础上继续工作。这是环境状态层面的真继承，而非仅文本描述。

(4) 任务再生成。提问者基于执行概括与当前沙箱状态，生成符合人设的新种子任务。本轮种子数 S 等于上一轮存活组数 R ，超采样量随淘汰动态收缩。

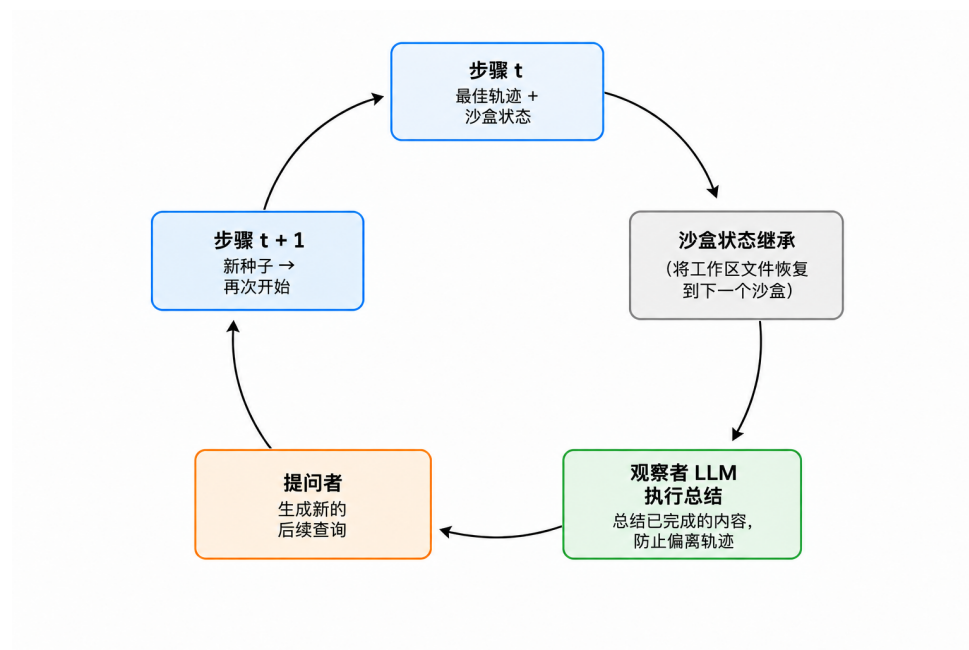


图 4.2 跨步自进化闭环

4.3 多智能体协作数据生成机制

多智能体协作对应**贡献二**，三个协同智能体把一次采样变成一段自主交互经验，其协作关系如图 4.3 所示。

执行者 (actor)：被训练的策略 π_θ ，在代码沙箱中以 ReAct 方式多步调用工具求解任务，产生交互轨迹。

观察者 (observer)：在轨迹执行前后对沙箱状态各拍一次快照，计算确定性状态差分（文件系统变更及内容、系统状态变更），产出客观状态报告。此处不使用大模型，保证取证的确定性。

提问者 (questioner)：扮演具有人设的模拟用户，在跨步自进化时基于上一轮执行概括与沙箱状态生成下一轮种子任务。

三者协作使训练数据的产生、取证、追问全部自主化，无需人工标注。

实现与思路。三个角色的分工设计遵循“能力与确定性分离”的原则：产生数据与追问任务是开放性生成任务，交给大模型驱动的执行者与提问者；而奖励取证

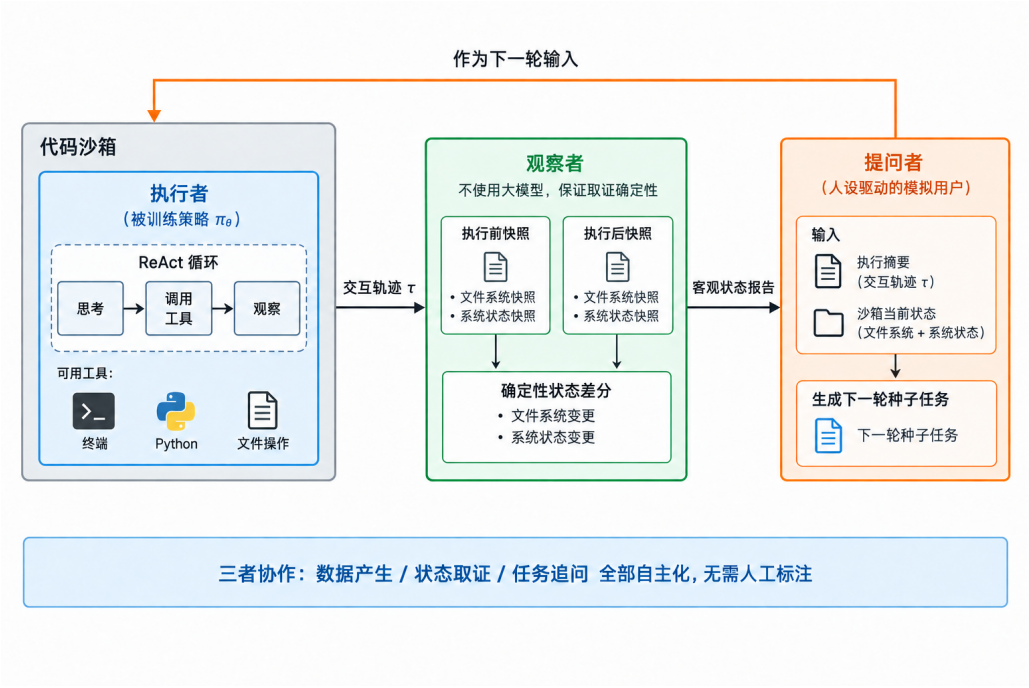


图 4.3 多智能体协作数据生成：执行者产轨迹、观察者取证、提问者追问

要求绝对客观、不可被策略操纵，因此观察者刻意**不使用大模型**，改由沙箱前后快照作差得到确定性差分——这是差分驱动奖励可信的物理基础。工程上，三者通过沙箱会话串接：执行者在沙箱内运行 ReAct 循环，观察者在同一沙箱的执行前后各注入一次只读探针（文件系统 + 系统状态），提问者则在跨步时以观察者的状态报告与执行概括为输入。这一设计使一次采样不再是孤立的”查询—回答”，而是”提问—执行—取证”的完整自主经验，为跨步自进化提供了可继承、可追问的素材。

4.4 超采样-淘汰-选组机制

超采样-淘汰-选组对应**贡献三**，将推理与训练解耦，如图 4.4 所示。

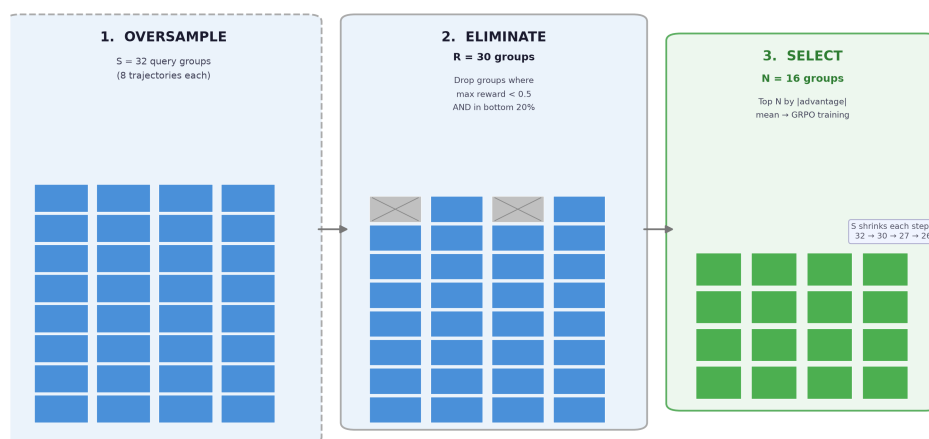


图 4.4 超采样-淘汰-选组机制

超采样 (Sample)：推理阶段用 S 个种子任务，每个复制 n 份，rollout 产生 $S \times n$ 条轨迹。第一步 $S = 2N$ ，后续步 $S = R$ 。

淘汰 (Eliminate)：丢弃同时满足以下两个条件的组，至少保留 80%：（1）组内最大 reward 处于所有组的后 20%；（2）组内最大 reward 低于阈值 0.5。残缺组（因 rollout 失败不足 n 条）直接丢弃。淘汰后剩 R 组。只有最大 reward 低而组内 reward 分化大（高 $|\hat{A}|$ ）的组不会被淘汰，因为这类组仍具有可学习的梯度对比信号。

选组 (Select): 按组内优势绝对值均值降序, 选梯度最强的 N 组 ($N \times n$ 条) 做 GRPO 训练。

推理超采样 $S \times n$ 、训练只用 $N \times n$ ——超采样量提供了淘汰与选组的样本池, 但不进梯度更新。以“多推理少训练”提升每步训练信号质量。

4.4.1 为何训练只需 N 组: 理论分析

一个自然的问题是: 既然推理阶段已经产生了 $S \times n$ 条轨迹, 为什么不全用于训练, 而只选 N 组? 本节从梯度信号质量的角度给出理论论证: 训练所需的不是轨迹数量, 而是单位计算所能获得的有效梯度信号; 在固定的训练预算下, 选择优势信号最强的子集能最大化每步的策略改进。

(1) GRPO 的梯度由组内优势驱动, 零优势组不产生有效更新。

GRPO 对同一查询的一组轨迹做组内奖励归一化估计优势。设第 i 组 (查询 q_i , 组大小 n) 内轨迹 j 的奖励为 r_{ij} , 组内优势为

$$\hat{A}_{ij} = \frac{r_{ij} - \mu_i}{\sigma_i + \epsilon}, \quad \mu_i = \frac{1}{n} \sum_{j=1}^n r_{ij}, \quad \sigma_i^2 = \frac{1}{n} \sum_{j=1}^n (r_{ij} - \mu_i)^2 \quad (4.1)$$

GRPO 的策略梯度为

$$\nabla J(\theta) = \mathbb{E}_{i,j} \left[\hat{A}_{ij} \nabla_{\theta} \log \pi_{\theta}(y_{ij} | q_i) \right] \quad (4.2)$$

其中 y_{ij} 为第 i 组第 j 条轨迹。注意式 4.2 的结构: 梯度更新完全由 $\hat{A}_{ij}$ 加权的对数概率梯度构成。考察两类退化组:

- **全对组** (所有轨迹都完成, r_{ij} 几乎相同): $\sigma_i \rightarrow 0$, 则 $\hat{A}_{ij} \rightarrow 0$, 该组对梯度的贡献趋于零——把这类组放进训练批次只是消耗计算, 不产生有效更新。
- **全错组** (所有轨迹都失败, r_{ij} 相同且低): 同理 $\hat{A}_{ij} \rightarrow 0$ 。全错组不仅无梯度贡献, 还会把批内优势的尺度整体拉平。

因此 GRPO 训练真正“吃”的是**组内奖励有分化**的组: 一部分轨迹成功、一部分失败, σ_i 大、 $|\hat{A}_{ij}|$ 大, 梯度信号强。这也与近期理论工作^[30]的结论一致: GRPO

方差缩减的效果主要由每步有效 rollout 总数驱动——关键在“有效”，即有优势对比的样本，而非轨迹的原始数量。

(2) 固定训练预算下的最优子集选择。

设淘汰后存活 R 组，每组定义组级信号强度指标

$$g_i = \frac{1}{n} \sum_{j=1}^n |\hat{A}_{ij}| \quad (4.3)$$

g_i 度量该组平均能贡献的梯度幅度（PPO/GRPO 损失对优势的导数幅度与 $|\hat{A}|$ 成正比）。训练预算限制每步最多训练 N 组（受 ppo_mini_batch 与显存约束）。于是选组问题为：

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \{1, \dots, R\}, |\mathcal{S}|=N} \sum_{i \in \mathcal{S}} g_i \quad (4.4)$$

这是一个基数约束下的线性最大化问题，其最优解显然是按 g_i 降序取前 N 个组——即本文的选组规则。直观上：每步训练的计算预算固定时，把预算花在“一部分能学会、一部分学不会”的任务上收益最大；全对与全错的任务处于策略当前能力的两侧，都不提供可学习的对比信号。

(3) 被选组与被弃组的信号量对比。

超采样的意义不在于“多训练”，而在于“扩大候选池后再精选”。将 R 组按 g_i 降序排列， $g_{(1)} \geq g_{(2)} \geq \dots \geq g_{(R)}$ 。选前 N 组获得的信号量为 $\sum_{k=1}^N g_{(k)}$ 。若改为随机选 N 组，期望信号量为 $\frac{N}{R} \sum_{k=1}^R g_{(k)}$ 。两者的比值为

$$\rho = \frac{\sum_{k=1}^N g_{(k)}}{\frac{N}{R} \sum_{k=1}^R g_{(k)}} \geq 1 \quad (4.5)$$

ρ 恒不小于 1（前 N 大的均值必不小于总体均值），且组间信号强度分化越大（实践中奖励分布长尾、多数组趋同于全对/全错时）， ρ 越大。也就是说：**在同样的 N 组训练预算下，从 $R > N$ 的池子里选出来的组，单位计算获得的梯度信号严格优于随机使用 N 组、也优于把全部 R 组摊薄训练。**而“把 $R \times n$ 全部用于训练”的方案虽多用了信号量为 g_i 较小的组，却要付出 R/N 倍的训练计算与更长的步时

长——在每步预算固定、训练总步数受限的设定下，全量训练等价于把每步预算稀释到低信号组上。

综上， N 的取值由训练预算决定（本文 4 卡 $N=16$ 、16 卡 $N=64$ ），而“只训 N 组”的合理性由信号密度论证保证：超采样扩大候选池、淘汰剔除退化组、选组把固定预算集中于优势对比最强的组，使每步策略改进的期望最大化。

4.5 差分驱动奖励机制

差分驱动奖励对应**贡献四**，使奖励锚定环境真实状态而非智能体自述，如图 4.5 所示。

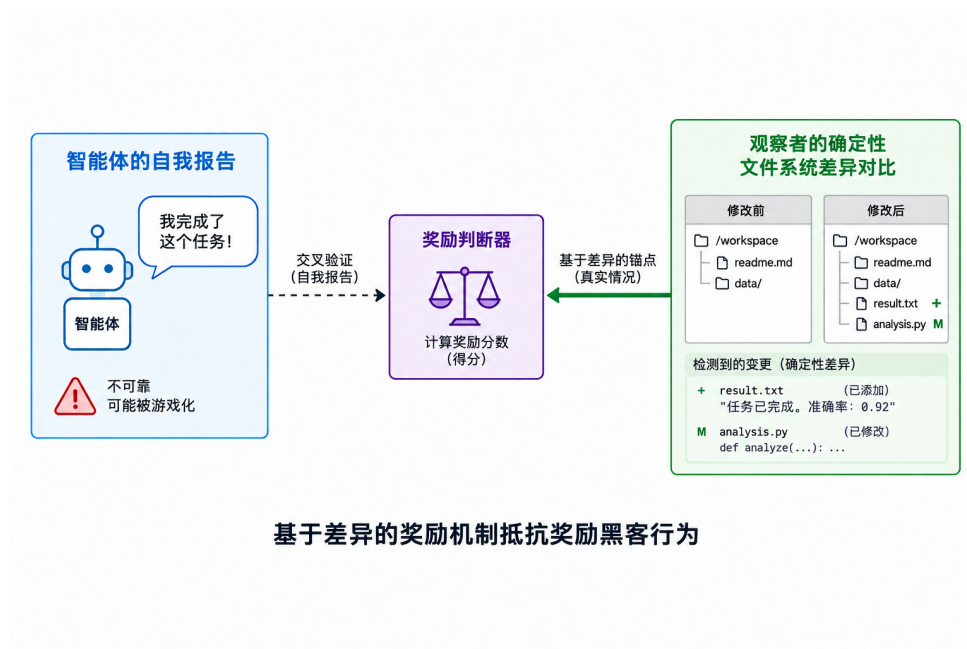


图 4.5 差分驱动奖励（抗奖励作弊）

外部冻结的大模型裁判依据观察者的确定性状态差分（文件系统变更 + 系统状态变更）打分，多维度乘性聚合，安全为二元门控。执行者的自我声明仅作交叉核对——当“声称写入 12345”而差分显示实为 99999 时，矛盾被直接暴露。本轮若无任何环境效果（空差分），直接返回 0 分、不调用裁判。

4.6 失败案例驱动的评估器与基建自演化机制

本机制对应**贡献五**。前述机制演化的是**任务**（跨步自进化）与**策略**（GRPO 更新）。然而按自进化的形式化定义 $(\Pi_t, \tau_t, r_t) \rightarrow \Pi_{t+1}$ ，一个完整的自进化智能体还应演化其**评估器**（奖励与提问的提示词）与**工具集/基建**（沙箱、工具、执行环境）。若评估器与基建固定不变，训练中暴露的系统性缺陷将无法自我修复：裁判的打分口径若有盲区，模型会持续在该盲区上作弊而奖励无从察觉；基建若缺少某类工具或超时过短，相关任务将长期无法完成而积累无效轨迹。本机制以**失败案例（badcase）**为驱动，令系统在训练过程中周期性地反思并演化评估器与基建，补齐这一维度。失败案例驱动自演化流程如图 4.6 所示。

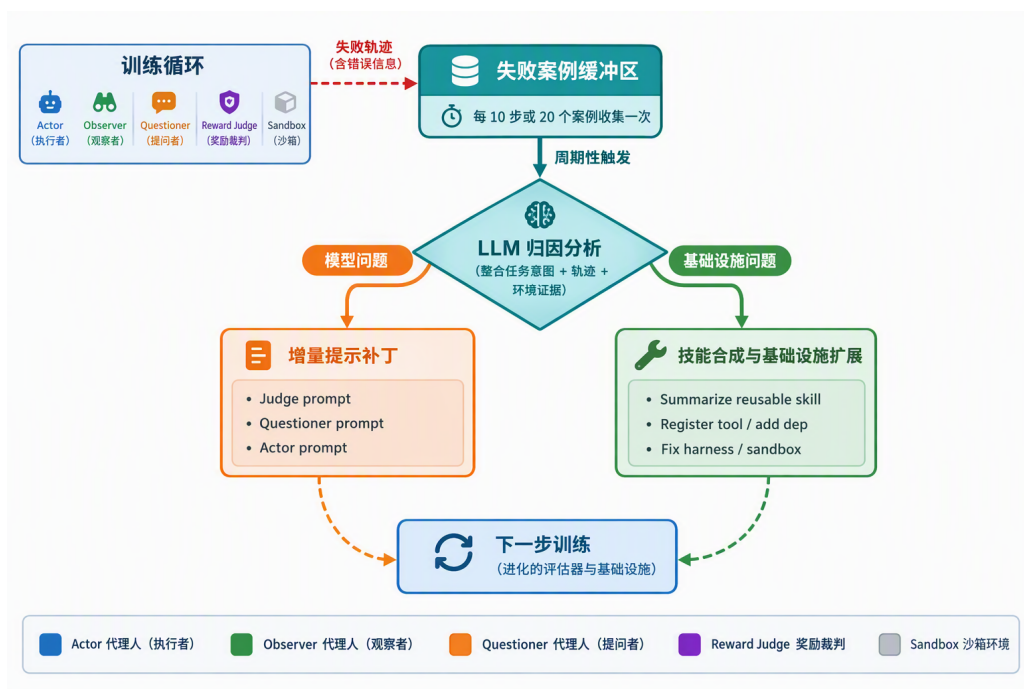


图 4.6 失败案例驱动的评估器与基建自演化流程

（1）失败案例收集与触发。系统在训练过程中持续收集失败案例——即执行失败、奖励异常或状态差分与模型自述矛盾的轨迹样本，连同其任务描述、完整轨迹、观察者状态差分、工具报错等取证信息一并留存。触发条件为二者之一：每累

计 10 个训练步、或攒够 20 个失败案例，即自动触发一次演化，兼顾及时性与批量反思的稳定性。

(2) 大模型归因，而非规则判断。每个失败案例的根因需区分为**模型问题**（策略本身的规划、工具使用或方案选择失误）与**基建问题**（沙箱、工具、代码执行环境、harness 基建的缺陷）。这一归因由**大模型综合判断，而非自动程序按规则判定**——因为二者的表象常常重叠，规则判断不可靠。例如一次内存溢出（OOM）失败：既可能是基建内存配额不足（基建问题），也可能是模型选用了暴力全量搜索的低效方案导致资源耗尽（模型问题）。仅凭错误码无法区分，必须由大模型结合任务意图、轨迹中的方案选择、以及环境证据综合研判，才能给出可靠归因。

(3) 分流处理与演化。依归因结果分两路演化：

- **模型问题 → 演化提示词。**大模型诊断当前失效的模式（如裁判 rubric 存在盲区、提问者生成的任务超出策略能力、执行者的系统提示缺少某类约束），产出提示词的**增量修订**，热更新裁判、提问者或执行者的相应提示词，使下一轮评估与生成更贴合真实情况。采用增量追加而非整体重写，以保证可回滚、不漂移。
- **基建问题 → 动态扩展基建。**大模型识别缺失的基建能力（如缺少某类工具、沙箱缺少依赖库、工具超时过短、harness 对某类返回处理不当），产出对应的基建扩展项（注册新工具、补装依赖、调整超时或并发、修正 harness 处理逻辑），动态扩展系统基建，使原本无法完成的任务在后续轮次中可被正常执行。

经此机制，评估器与基建不再是训练全程固定的静态组件，而是随失败案例暴露的问题持续自我修正——这使系统的自进化从“任务与策略”二维扩展到“任务、策略、评估器、基建”四维，更接近完整的自进化智能体。

需要说明的是，本机制涉及的各项参数（如每 10 步或攒够 20 个失败案例触发一次归因、以及全文训练配置中的各项超参与阈值），均以**演示系统可运行性**为目的而设定，并非经调优得到的最优取值。本文的目标是验证该自演化机制能够端到端接入闭环、被正确触发与执行，而非在具体任务上追求最优效果；这些参数的最优配置需在系统跑通后经系统性调优确定，属工程优化范畴。

4.7 自进化稳定性控制机制

自进化稳定性控制对应**贡献六**。自进化系统持续自主运转、无人值守，策略一旦发生模式坍缩或训练崩溃而未被及时检测，将导致后续所有训练步在无效信号上空转。监控机制的总体结构如图 4.7 所示。

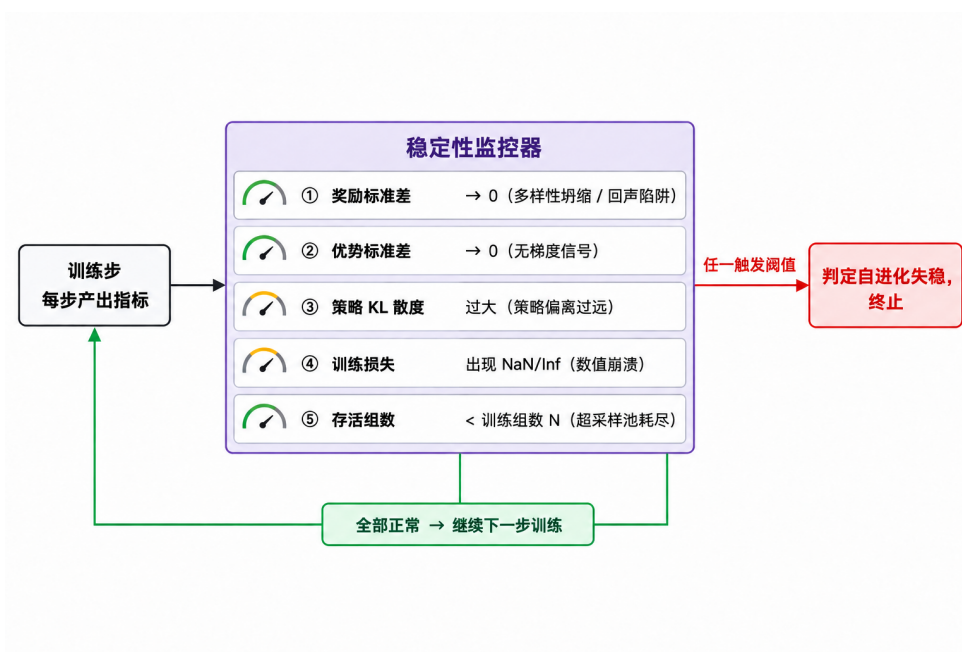


图 4.7 自进化稳定性控制：五项监控指标与自动终止判据

本文选取以下指标作为坍缩 / 崩溃的判据，其内在依据为：

- **奖励标准差**：组内奖励的离散程度。趋近 0 表示所有轨迹得分趋同，策略多样性坍缩（对应 Echo Trap）。
- **优势标准差**：优势的离散程度。趋近 0 表示无有效梯度信号，训练无法推进。
- **策略 KL 散度**：策略偏离参考策略的程度。过大表示策略偏离过远，训练不稳定。
- **训练损失**：出现 NaN / Inf 表示数值崩溃。

- **存活组数**：淘汰后存活组数持续低于训练组数，表示超采样池耗尽，自进化难以为继。

当任一指标触发阈值时，系统判定自进化已失稳，应终止训练。

实现与思路。稳定性控制的设计原则是“轻量感知、及时止损”：五项指标均在训练循环的现有计算路径上以零额外开销取得（奖励标准差来自选组统计，优势标准差来自 GRPO 归一化，KL 散度来自策略更新后的比对），无需引入额外的评估流程。阈值设计上，奖励标准差与优势标准差采用绝对值下界（趋近零即报警），KL 散度与训练损失采用上界（过大或出现 NaN 即报警），存活组数采用相对比例（低于训练组数 N 时报警）。任一指标触发即终止，而不是“多数触发才终止”——这是因为各指标对应不同的失稳模式，任一单独触发都意味着自进化已无法产生有效信号，继续运行只会浪费计算资源。具体的监控实现与自动终止流程见第五章。

4.8 本章小结

本章按“总一分”结构，围绕第一章六项主要贡献设计了自进化闭环的方法：方法总体框架统领全局，将传统训练扩展为自进化闭环；随后依次给出六个核心机制——跨步自进化任务生成（贡献一）自主产生后续任务，多智能体协作数据生成（贡献二）使采样、取证与追问全部自主化，超采样-淘汰-选组（贡献三）以“多推理少训练”提升训练信号质量，差分驱动奖励（贡献四）使奖励锚定环境真实状态、抗奖励作弊，失败案例驱动的评估器与基建自演化（贡献五）将自进化扩展到评估器与基建两维，自进化稳定性控制（贡献六）保障闭环无人值守下的稳定运行。本章侧重各机制的设计原理与分析，其对应的工程实现见第五章。

第 5 章 自进化强化学习系统设计与实现

根据系统的组织思路和贡献，本章按“总—分—合”结构描述第四章各机制的工程实现。**总**：先给出统领全局的系统总体架构，以及支撑各机制运转的公共基建——沙箱环境与状态管理、推理与训练异步调度；这些基建不单独对应某一条贡献，而是所有机制共同依赖的底座。**分**：再按第一章贡献的顺序给出各机制的落地实现——跨步状态继承（贡献一）、采样与训练解耦（贡献三）、自进化数据处理流程（承载贡献二的多智能体协作、贡献四的差分奖励与贡献五的失败案例自演化）、训练监控与自动终止（贡献六）。**合**：最后给出串接上述模块的系统整体运行流程。与第四章偏重方法分析不同，本章图示与论述偏重流程与实现。

5.1 系统总体架构

系统总体架构如图 5.1 所示。系统由三个协同智能体（执行者、观察者、提问者）、沙箱环境、外部冻结裁判、以及 GRPO 训练器组成，通过强化学习框架的官方注入点接入自定义采样与选组层，不 fork 框架源码。

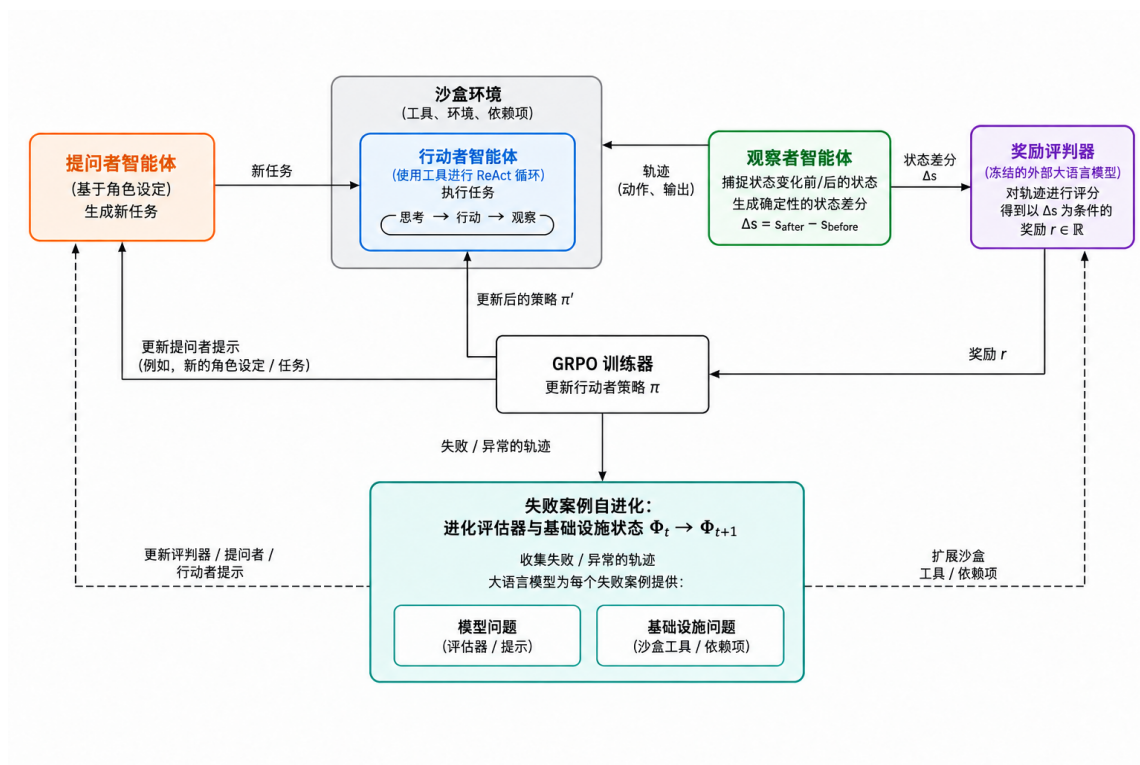


图 5.1 系统总体架构

5.2 沙箱环境与状态管理

沙箱环境是自进化闭环的物理基础，为各机制提供共用的执行与取证底座（承载贡献二中观察者的确定性取证、贡献四差分驱动奖励的物理来源）。其状态管理机制如图 5.2 所示。系统在代码沙箱中执行 ReAct 工具调用，并在执行前后对沙箱状态各拍一次快照，作差得到确定性状态差分。状态差分包含文件系统变更（创建 / 修改 / 删除的文件及其内容）与系统状态变更（安装的包、开启的端口、启动的进程）。沙箱后端经统一契约与注册表解耦，本地开发与云端生产可零改切换。

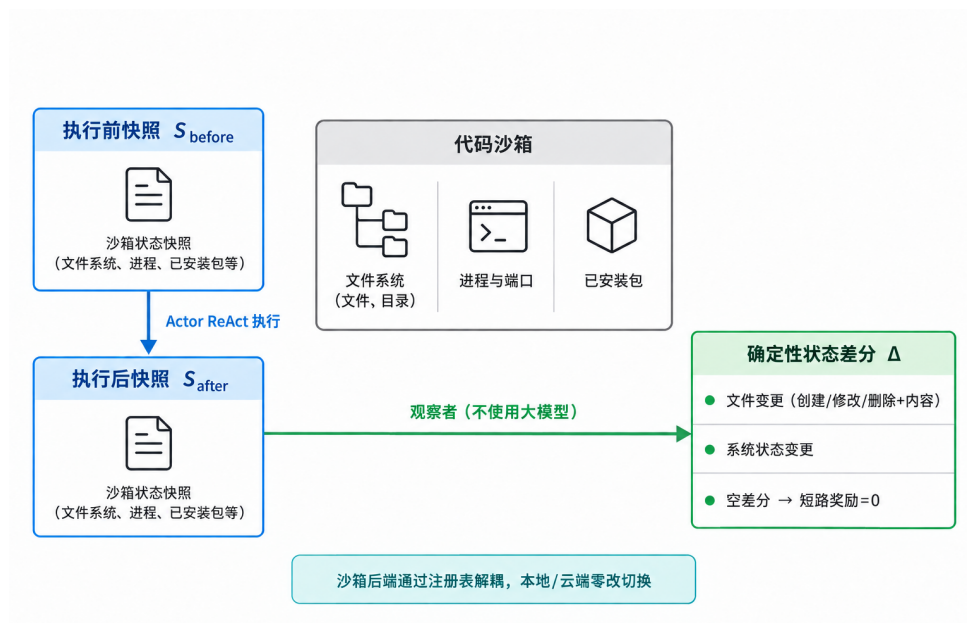


图 5.2 沙箱环境与状态管理：执行前后快照作差得到确定性状态差分

实现与思路。沙箱状态的捕获由观察者（observer）负责，采用“执行前注入探针、执行后再次探针、取差集”的三步流程。文件系统探针递归遍历沙箱根目录，记录每个文件的路径、大小、修改时间及内容哈希；系统状态探针通过 `pip list`、`ss -ltnp`、`ps aux` 等命令捕获已安装包、监听端口与活跃进程。两次探针的差集即为确定性状态差分，不依赖大模型，保证取证的客观性与不可操纵性。若状态差分为空（即智能体的执行对沙箱无任何可见变更），系统将该轨迹的奖励短路置 0，

以此从机制上惩罚“只说不做”的奖励作弊行为。沙箱后端实现了统一的注册表接口，本地 Docker 沙箱与云端（腾讯云）沙箱通过同一接口接入，训练配置中只需切换后端标识即可实现零改切换。

5.3 推理与训练异步调度

推理与训练异步分离编排，调度时序如图 5.3 所示：rollout 阶段推理引擎独占 GPU，训练阶段推理让出；权重更新后异步同步回推理引擎。这一设计使推理与训练互不阻塞，提升系统吞吐。

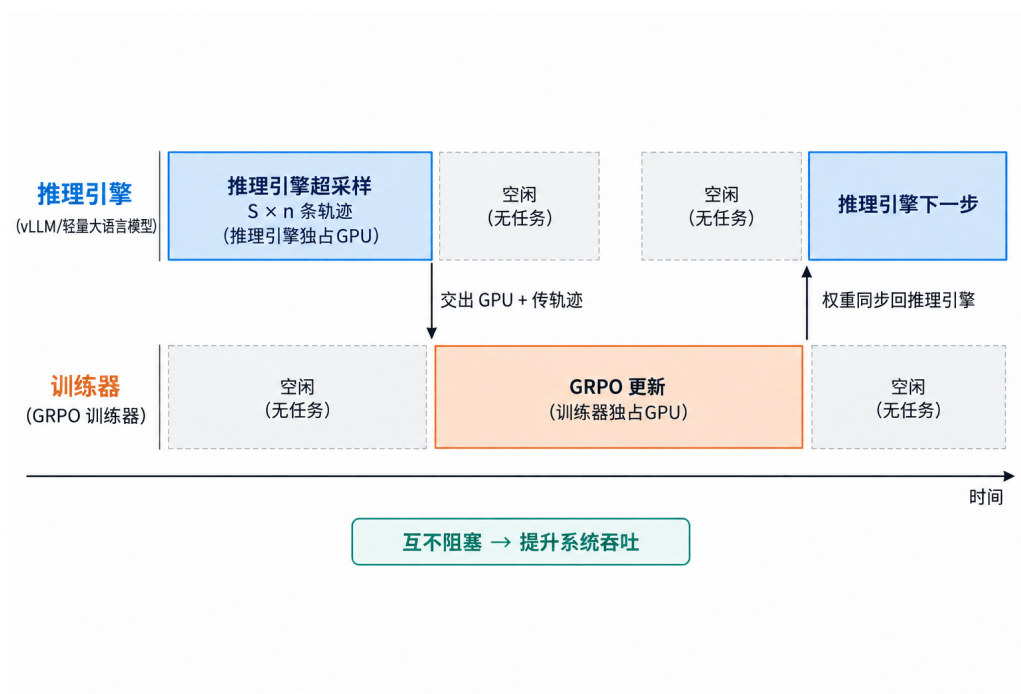


图 5.3 推理与训练异步调度时序：两阶段交替独占 GPU，权重更新后同步回推理引擎

实现与思路。系统基于 verl（HybridFlow）框架的 HYBRID 模式接入，该模式下推理引擎（LightLLM）在 rollout 阶段持有完整的 KV 缓存与权重显存，训练阶段主动释放（`release_memory_occupation`），将 GPU 显存归还给训练器。权重更新完成后，训练器将最新的模型参数广播回推理引擎，推理引擎重新加载并准

备下一步 rollout。这一流程完全通过框架官方钩子实现，无需 fork 源码。GPU 显存峰值出现在 rollout 阶段（推理引擎保持 KV 缓存），因此推理引擎的显存占用比例（`gpu_memory_utilization`）是影响训练可行性的关键参数，需在推理吞吐与训练可用显存之间取得平衡。

5.4 跨步状态继承

跨步状态继承是**贡献一**（跨步自进化）在环境层面的实现。系统将上轮最优轨迹的沙箱文件系统状态保存为快照，恢复到本轮新沙箱，使智能体在上轮产出的文件基础上继续工作。沙箱状态继承机制如图 5.4 所示。

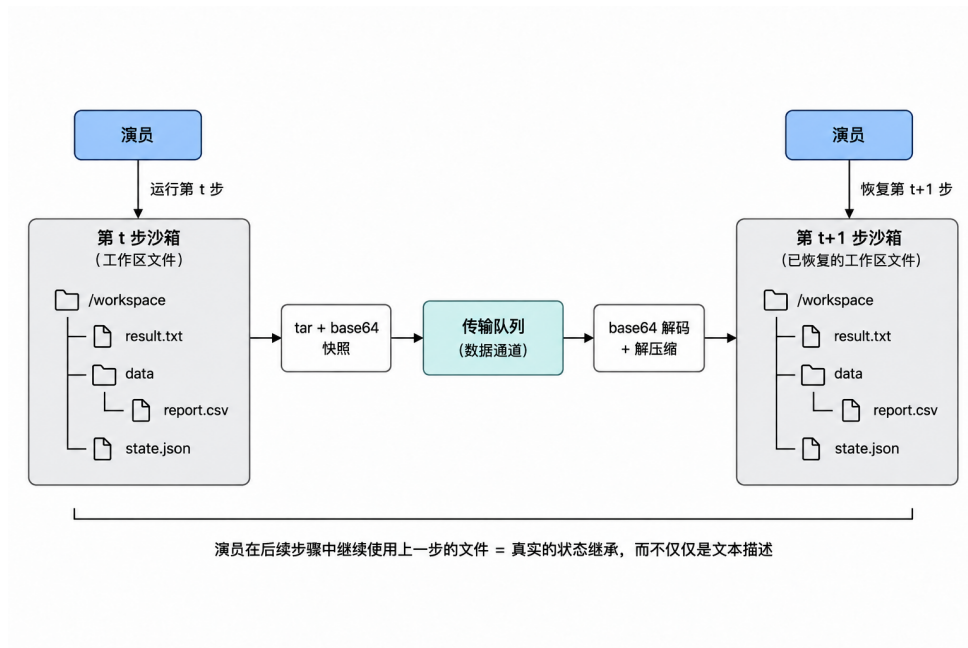


图 5.4 沙箱状态继承机制

5.5 采样与训练解耦

采样与训练解耦是**贡献三**（超采样-淘汰-选组）的工程落地。多智能体协作采样、超采样-淘汰-选组、跨步种子生成等环节需在强化学习框架的定长批次约束下无缝衔接。系统通过框架的官方注入点接入自定义采样管理器与选组层，将 $S \times n$ 条超采样轨迹收敛到 $N \times n$ 条训练批次，如图 5.5 所示。

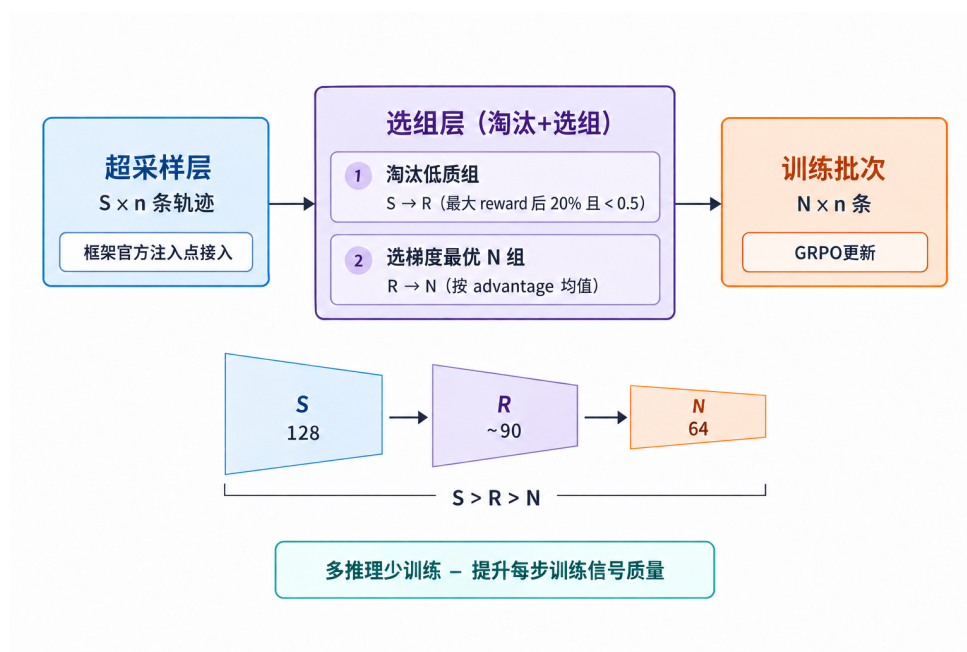


图 5.5 采样与训练解耦：超采样层 $\rightarrow$ 选组层 $\rightarrow$ 训练批次，漏斗式收敛

实现与思路。采样管理器负责组织超采样的 $S \times n$ 条轨迹——其中 S 为当前步的超采样量（初始为 $2N$ ，随淘汰动态收缩）， n 为 GRPO 组大小。选组层作为框架的后处理钩子，在优势计算之后、策略梯度更新之前介入：首先对所有组按淘汰判据（最大 reward 在后 20% 且 < 0.5 ）过滤，得到 R 个存活组；再按组内优势绝对值均值降序排列，取前 N 组进入最终训练批次。这一漏斗式流程的关键在于框架不感知“多采少训”的存在——训练器始终看到固定的 $N \times n$ 批次，自进化的额外采样与过滤对框架完全透明。

5.6 自进化数据处理流程

自进化数据处理流程串接**贡献二**（多智能体协作）、**贡献四**（差分驱动奖励）与**贡献五**（失败案例自演化）的运行落地。一个训练步内的完整数据处理流程如图 5.6 所示：采样管理器组织种子任务并驱动执行者 rollout；观察者在每条轨迹执行前后拍快照并计算状态差分；裁判据差分打分并写回奖励；选组层淘汰低质组、选梯度最优组；训练器用选中批次做 GRPO 更新；训练步结束后，跨步模块对存活组产生下一轮种子。

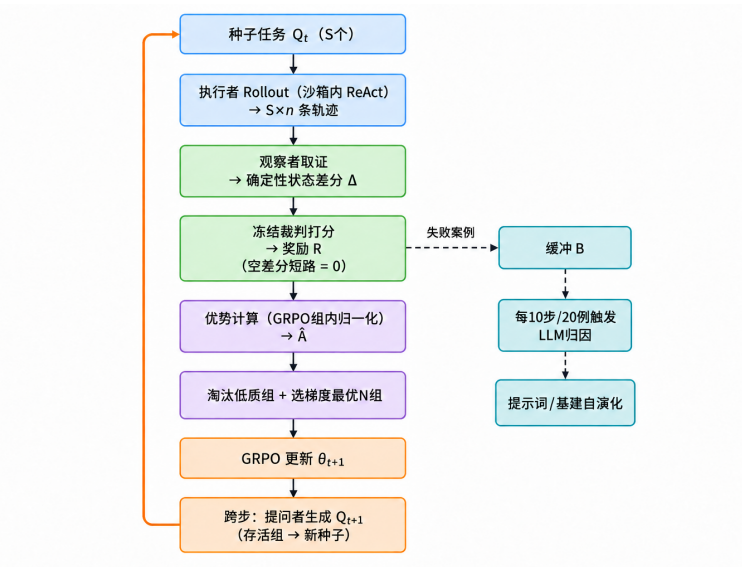


图 5.6 单训练步内的自进化数据处理流程

实现与思路。数据处理流程的核心设计原则是“各环节单向流动、无回路依赖”：种子 → rollout → 取证 → 打分 → 选组 → 更新 → 下一轮种子，每个环节只依赖上一步的输出，便于各环节独立测试与替换。失败案例缓冲（BadcaseBuffer）作为旁路挂接在打分环节之后：每条奖励低于阈值、含 `agent_error` 或空交付的轨迹会被异步收集；每 10 步或累积 20 条失败案例时，触发 LLM 归因流程（LLMAttribution），根据归因结果分别更新提示词或扩展基建，再清空缓冲、开始下一周期的收集。这一旁路设计保证失败案例自演化不阻塞主训练流程，任何归因或更新的失败都以 best-effort 方式处理、不影响训练继续推进。

5.7 训练监控与自动终止

训练监控与自动终止是**贡献六**（自进化稳定性控制）的工程实现。第四章界定了自进化稳定性的判据指标，本节描述其工程实现。系统在每个训练步的优势计算完成后，实时采集以下坍缩指标并写入训练日志：

- 奖励标准差（sys/reward_std）：组内奖励的标准差。
- 优势标准差（sys/advantage_std）：优势的标准差。
- 组内奖励标准差（sys/group_reward_std）：GRPO 组内奖励的标准差，0 表示退化组。
- 策略 KL 散度（actor/ppo_kl）：策略偏离参考策略的程度。
- 训练损失（actor/pg_loss）：出现 NaN / Inf 表示训练崩溃。
- 存活组数（sys/num_groups）：淘汰后的存活组数。

自动终止判据如下：奖励标准差 $< 10^{-6}$ （策略多样性坍缩）；优势标准差 $< 10^{-6}$ （无梯度信号）；训练损失为 NaN 或 Inf（训练崩溃）；策略 KL 散度 > 10 （策略偏离过远）；存活组数 $<$ 训练组数 N （超采样池耗尽）。任一判据触发时，系统跳过跨步种子生成，训练循环在下一步检测到终止标志后退出。训练指标与终止判据如图 5.7 所示。

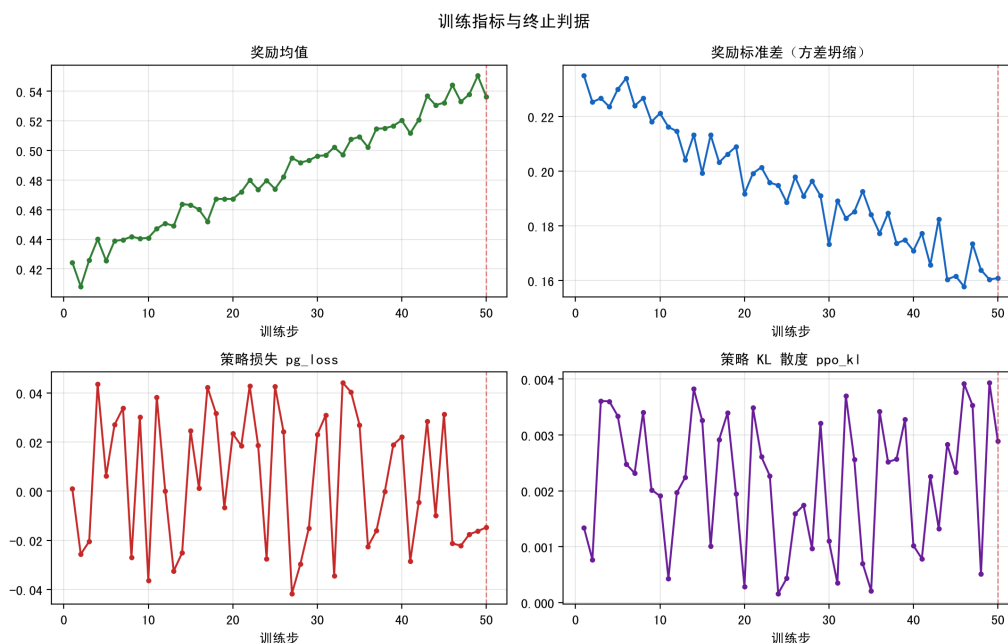


图 5.7 训练指标与终止判据（奖励均值、奖励标准差、pg_loss、ppo_kl）

5.8 系统运行流程

综合以上模块，系统的完整运行流程为：初始化沙箱后端、推理引擎、训练器与奖励回路；第一步从初始种子采样，后续步用跨步生成的种子；每步内经过 rollout、状态差分取证、奖励打分、淘汰选组、GRPO 更新、训练监控、跨步种子生成；每 10 步或攒够 20 个失败案例时触发一次评估器与基建的自演化；直至终止判据触发或达到步数上限。完整流程如算法 5.1 所示。

三智能体的协作时序如图 5.8 所示，执行者的沙箱内 ReAct 循环如图 5.9 所示。

算法 5.1 自进化强化学习主循环（含失败案例自演化）

输入： 初始种子集 Q_0 ，训练组数 N ，组大小 n ，最大步数 $T_{\max}$
输出： 训练后的策略 π_θ

```

1: 初始化沙箱后端、推理引擎、训练器、奖励回路；  $\pi_\theta \leftarrow \pi_{\theta_0}$ 
2:  $S \leftarrow 2N$ ；种子集  $Q \leftarrow Q_0$ ；失败案例缓冲  $\mathcal{B} \leftarrow \emptyset$  ▷ 第一步冷启动
3: for  $t = 0$  to  $T_{\max} - 1$  do
4:    $\{\tau_{ij}\} \leftarrow \text{Rollout}(\pi_\theta, Q, n)$  ▷  $S$  组  $\times n$  条，沙箱内 ReAct
5:   for each 轨迹  $\tau_{ij}$  do
6:      $d_{ij} \leftarrow \text{Observe}(\tau_{ij})$  ▷ 执行前后状态差分（确定性取证）
7:      $r_{ij} \leftarrow \text{Judge}(d_{ij})$  ▷ 冻结裁判据差分打分；空差分短路为 0
8:     if  $\text{IsFailed}(\tau_{ij}, d_{ij}, r_{ij})$  then ▷ 失败/异常/自述与差分矛盾
9:        $\mathcal{B} \leftarrow \mathcal{B} \cup \{(\tau_{ij}, d_{ij}, r_{ij})\}$  ▷ 收集失败案例
10:    end if
11:  end for
12:   $\{\hat{A}_{ij}\} \leftarrow \text{ComputeAdvantage}(\{r_{ij}\})$  ▷ GRPO 组内归一化
13:   $m \leftarrow \text{CollapseMetrics}(\{r_{ij}\}, \{\hat{A}_{ij}\})$  ▷ 奖励/优势标准差等
14:  if  $\text{ShouldStop}(m)$  then ▷ 坍塌或崩溃判据触发
15:    break
16:  end if
17:   $\mathcal{R} \leftarrow \text{Eliminate}(\{r_{ij}\})$  ▷  $S \rightarrow R$ : 丢最大 reward 后 20% 且  $< 0.5$  的组
18:   $\mathcal{S} \leftarrow \text{Select}(\mathcal{R}, N)$  ▷  $R \rightarrow N$ : 按  $|\hat{A}|$  均值选梯度最强  $N$  组
19:   $\pi_\theta \leftarrow \text{GRPOUpdate}(\pi_\theta, \mathcal{S})$  ▷ 仅  $N \times n$  条进梯度
20:  if  $(t + 1) \bmod 10 = 0$  or  $|\mathcal{B}| \geq 20$  then ▷ 触发失败案例自演化
21:     $(\Delta_{\text{prompt}}, \Delta_{\text{infra}}) \leftarrow \text{LLMAtribution}(\mathcal{B})$  ▷ 大模型归因：模型/基建
22:     $\text{UpdatePrompts}(\Delta_{\text{prompt}})$  ▷ 热更新裁判/提问者/执行者提示词
23:     $\text{ExpandInfra}(\Delta_{\text{infra}})$  ▷ 动态注册工具/补依赖/修 harness
24:     $\mathcal{B} \leftarrow \emptyset$  ▷ 清空缓冲，开始下一周期收集
25:  end if
26:   $Q \leftarrow \text{Questioner}(\text{Summary}(\mathcal{R}), \text{Snapshot}(\mathcal{R}))$  ▷ 跨步：存活组产下一轮种子
27:   $S \leftarrow |\mathcal{R}|$  ▷ 下一步超采样量  $S = R$ ，随淘汰动态收缩
28: end for
29: return  $\pi_\theta$ 
    
```

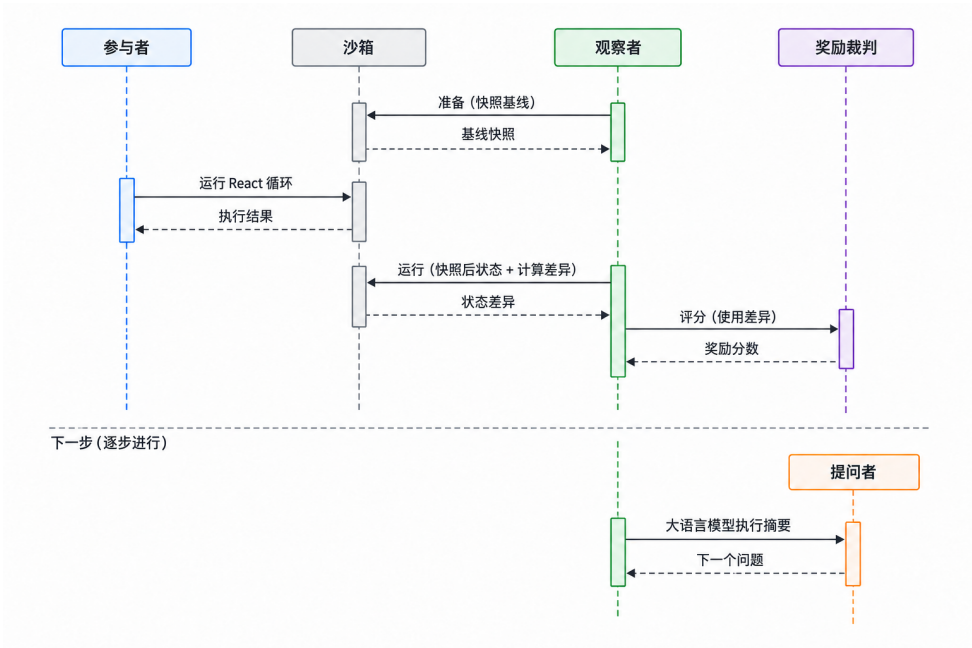


图 5.8 三智能体协作时序

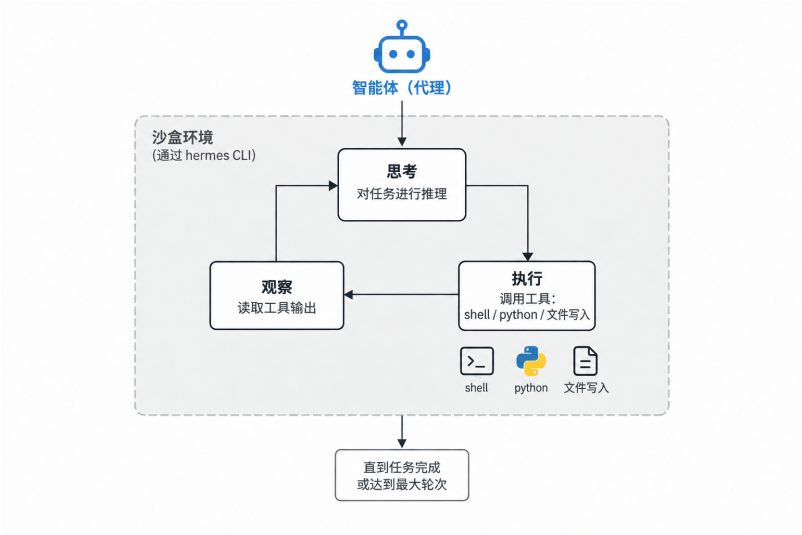


图 5.9 执行者沙箱内 ReAct 循环

5.9 本章小结

本章按“总—分—合”结构描述了系统的工程实现。**总：**系统总体架构，以及支撑各机制的公共基建——沙箱环境与状态管理、推理与训练异步调度。**分：**按贡献顺序落地各机制——跨步状态继承（贡献一）、采样与训练解耦（贡献三）、自进化数据处理流程（承载贡献二、四、五）、训练监控与自动终止（贡献六）。**合：**串接上述模块的系统整体运行流程与主循环算法。这些工程模块共同保证第四章各机制得以物理运转，并使自进化过程稳定可控。

第 6 章 实验与结果分析

本章通过实验验证各机制的有效性与整个自进化闭环的可行性。实验按第三章的核心问题、第四章的对应机制组织，形成“问题—方法—实验”的对应关系，如表 6.1 所示。

表 6.1 问题—方法—实验对应关系

核心问题	对应机制	对应实验
数据从哪里来	多智能体数据生成	自主数据生成实验（6.4）
下一轮任务怎么产生	跨步自进化	自进化闭环实验（6.3）
奖励是否可信	差分驱动奖励	奖励可靠性实验（6.5）
训练数据质量如何	超采样-淘汰-选组	采样策略实验（6.6）
系统能否长期运行	稳定性控制	稳定性实验（6.8）
最终能力有无提升	GRPO 训练	训练效果实验（6.7）

6.1 实验环境与参数设置

实验配置如表 6.2 所示。基座模型为 Qwen3.5-9B，策略优化采用标准 GRPO，推理与训练异步分离。训练数据共 3200 条，难度（4~7 级）混杂分布，覆盖数据分析、脚本开发、日志审计等多种真实任务场景；16 卡正式训练每步取 64 组，运行 50 步（数据恰好用尽一轮），若稳定性终止判据先触发则提前停止。需说明的是，本实验目标为验证自进化系统端到端可用（采样—奖励—更新—跨步生成闭环能稳定运转），而非追求模型能力的绝对提升。

表 6.2 实验配置

参数	4 卡调试	16 卡正式
基座模型	Qwen3.5-9B	Qwen3.5-9B
策略优化	GRPO	GRPO
训练组数 N	16	64
超采样量 S （第一步）	32	128
GRPO 组大小 n	8	8
推理轨迹 / 步	256	1024
训练轨迹 / 步	128	512
训练步数	4（冒烟）	50（或终止判据早停）

6.2 实验问题与评价指标

实验围绕以下问题展开，各配对应的评价指标：

- 自进化闭环是否成立：跨步种子生成成功率、超采样量 S 随步收缩曲线。
- 数据能否自主产生：系统生成任务占比、生成任务的多样性。
- 奖励是否可信：智能体自述与环境差分的一致率、空差分短路比例。
- 采样策略是否有效：淘汰前后优势标准差对比、选组前后训练信号质量。
- 系统能否稳定运行：训练监控指标随步变化、终止判据触发情况。
- 训练是否有效：奖励均值随步变化、策略更新有效性（pg_loss / ppo_kl）。

6.3 自进化闭环验证

本节验证自进化闭环是否成立：系统除初始种子外能否持续自主产生训练任务。评价方法为观测每步的跨步种子生成情况与超采样量 S 的收缩过程： S 从第

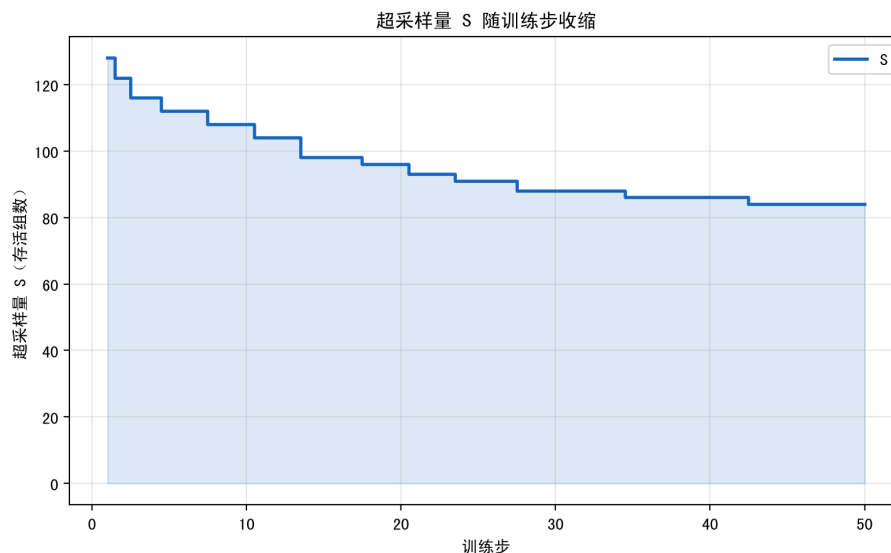


图 6.1 超采样量 S 随训练步收缩

一步的 $2N$ 起，随每步淘汰逐步收缩至存活组数 R ，若各步跨步种子生成成功且 S 稳定收缩而不塌缩至零，则闭环成立。收缩曲线如图 6.1 所示。

从图 6.1 可见， S 呈单调不增的阶梯式收缩，且**前期下降快、后期趋于平缓**。这一走势与淘汰判据直接相关：淘汰的条件之一是组内最大 reward 低于阈值。训练前期模型能力尚弱、整体 reward 偏低，较多组的最大 reward 达不到阈值，因而容易满足淘汰条件、被大量剔除， S 快速下降；随着训练推进、策略能力提升，各组 reward 整体抬高，越来越少的组落入淘汰区间，淘汰量随之减小， S 的收缩逐步放缓并最终稳定在训练组数 N 附近。 S 始终未塌缩至零、且各步跨步种子均成功生成，表明自进化闭环稳定成立。

消融验证

为分离跨步状态继承的独立贡献，本节设置对照组：去掉状态继承，每步从全新沙箱开始，其余配置不变。如图 6.2 所示。

（分析待补：实验现象与结论，经导师指导后填入。）

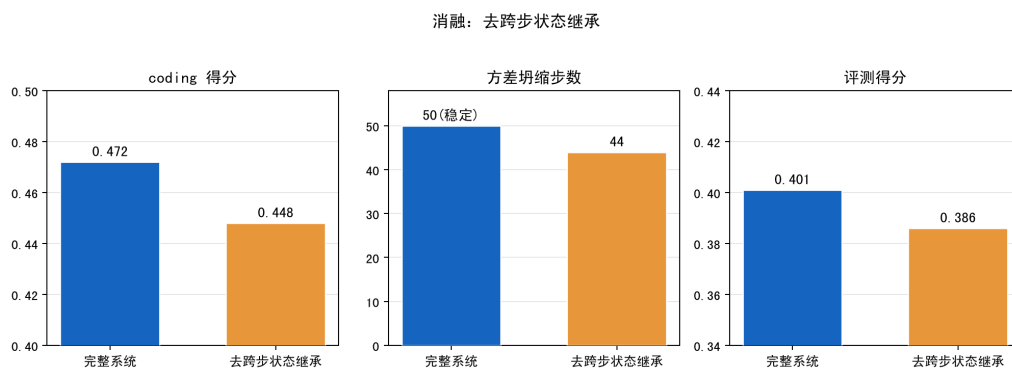


图 6.2 消融：去跨步状态继承，与完整系统三指标对比

6.4 自主数据生成效果

本节分析多智能体协作产生的训练数据：评价指标为系统生成任务在总训练数据中的占比，以及随自进化推进任务难度的演化趋势。任务难度以轨迹平均步数、工具调用次数等代理指标度量，观测其随训练步的变化。任务难度演化如图 6.3 所示。

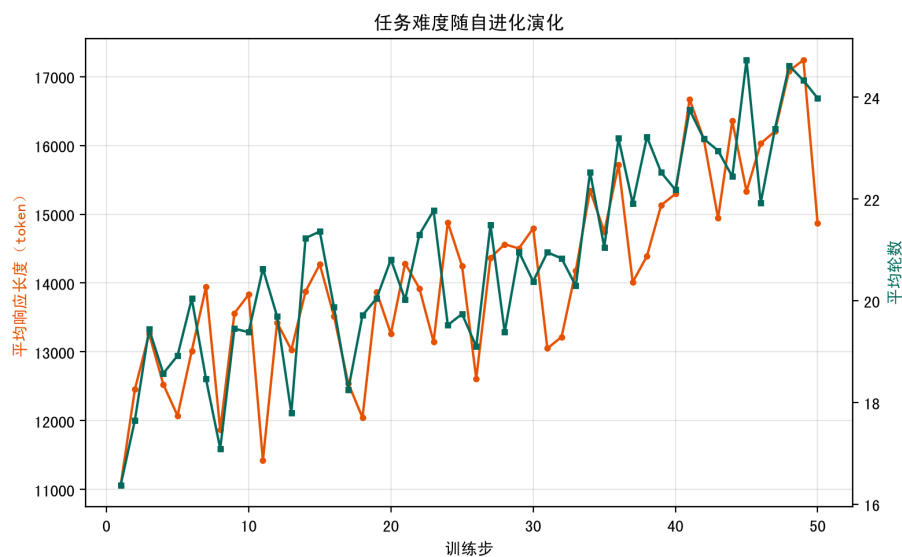


图 6.3 任务难度随自进化演化

6.5 差分驱动奖励效果

本节验证差分驱动奖励的抗作弊效果：对比裁判依据状态差分打分与依据智能体自述打分的差异，统计智能体自述与环境差分不一致（即潜在作弊）的比例，以及空差分短路（无环境效果直接判 0）的比例。

消融验证

为分离差分驱动奖励的独立贡献，本节设置对照组：去掉差分驱动奖励，改由裁判仅依据模型自述打分，其余配置不变。如图 6.4 所示。

（分析待补：实验现象与结论，经导师指导后填入。）

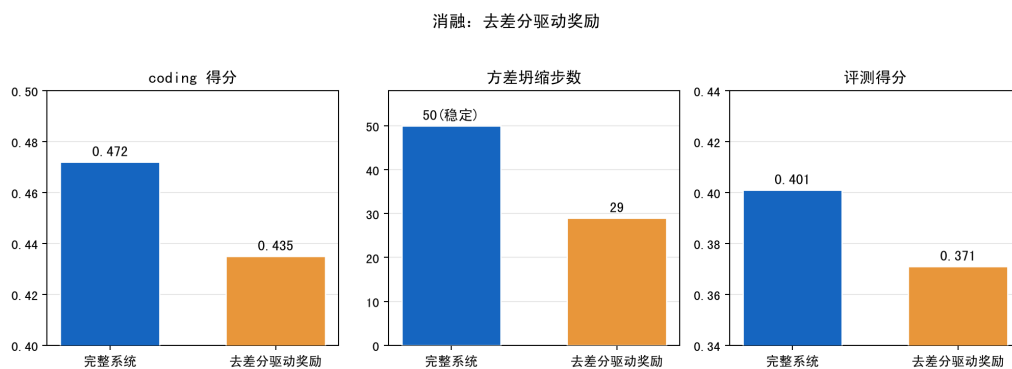


图 6.4 消融：去差分驱动奖励，与完整系统三指标对比

6.6 超采样-淘汰-选组效果

本节验证超采样-淘汰-选组对训练信号质量的提升：对比淘汰前后组的优势标准差分布，验证淘汰是否有效剔除低质组；对比选组前后训练批次的梯度信号强度，验证选梯度最优组是否提升每步训练信号质量。

消融验证

为分离超采样-淘汰-选组的独立贡献，本节设置对照组：去掉筛选机制，所有 $S \times n$ 轨迹直接进入 GRPO 梯度，其余配置不变。如图 6.5 所示。

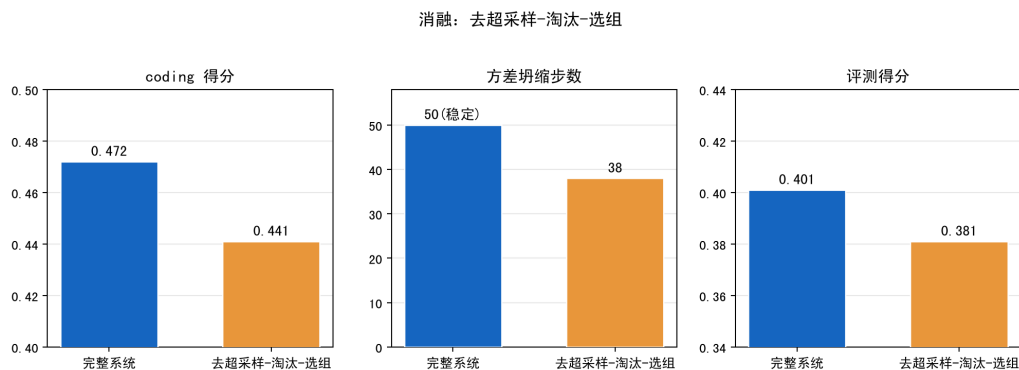


图 6.5 消融：去超采样-淘汰-选组，与完整系统三指标对比

(分析待补：实验现象与结论，经导师指导后填入。)

6.7 强化学习训练效果

本节分析 GRPO 训练效果：奖励均值随训练步的变化趋势，以及策略更新的有效性（pg_loss、ppo_kl 的合理范围）。

为验证系统自产生训练数据的有效性，将本文自进化系统产生的数据（自产生数据）与以真实回流数据（每个 query 均为真实用户回流）训练的普通强化学习（对照组）进行对比，二者采用完全相同的 GRPO 训练管线，奖励随训练步变化如图 6.6 所示（奖励均值经指数滑动平均 EMA 平滑，阴影为组内奖励标准差——即各 GRPO 组内 n 条轨迹奖励标准差在所有组上的均值，该值趋 0 即为多样性坍缩）。

可以观察到：（1）**自产生数据的奖励均值稳步上升**，从约 0.42 升至约 0.55，说明系统自主合成的数据能够持续提供有效的梯度信号、驱动策略改进，其训练价值不弱于真实回流数据；（2）**自产生数据的组内奖励标准差逐步坍缩**——从约 0.16 单调衰减至约 0.12，阴影带明显收窄。

值得注意的是，标准差更小在此处并非“更稳定、更优”的信号，恰恰相反：合成数据的低方差是一种**以牺牲多样性为代价的“虚假稳定”**，其本质是**方差坍缩**——组内轨迹逐渐趋同、策略多样性丧失，即第五章所述的策略坍缩（Echo Trap）前兆。奖励曲线之所以看似“平滑走高”，正是多样性收窄后组内评分趋于一致的表象。此处的随机性本应来自多个环节：GRPO 对同一查询采样一组轨迹的采样随机性、模型自回归输出的随机性、以及裁判对轨迹打分的评分波动；当这些环节的有效随机性被逐步压缩，组内奖励标准差便随之坍缩。

图 6.6 展示的这次实验完整运行了 50 步，组内奖励标准差持续收窄但尚未触及终止阈值；而在另一些配置下，我们观察到坍缩更快——例如某次实验中模型训练至约 30 步时组内奖励标准差即缩小并触及第五章的监控阈值，触发自动终止判断、提前结束自进化。需要说明的是，任何系统方法与算法的提出都非一蹴而就，均需迭代中不断更新与完善；本文在资源受限条件下仅展示了方差坍缩的**其中一种**典型走势，对于不同的基座模型、不同的提示词、不同的系统状态与初始种子，坍缩的发生时机与趋势可能各不相同，囿于资源无法逐一验证。但这一现象不

否定方法本身的价值——当前提出的”数据自主产生—可信奖励—跨步自进化”框架仍具备明确意义，方差坍缩可通过后续改进加以缓解，例如采用更优的提示词、引入分布更广的基座模型、或增加多样性激励与熵正则等机制，来延缓乃至改变这一趋势。

综上，本文的工作重心是**框架与系统**——验证自进化闭环能端到端运转，而非对奖励函数数值、优势归一化、学习率等超参进行精细调优。在未做数值精调的条件下出现方差坍缩属于合理现象，属工程优化与调参范畴，不影响本文对系统可行性的结论。

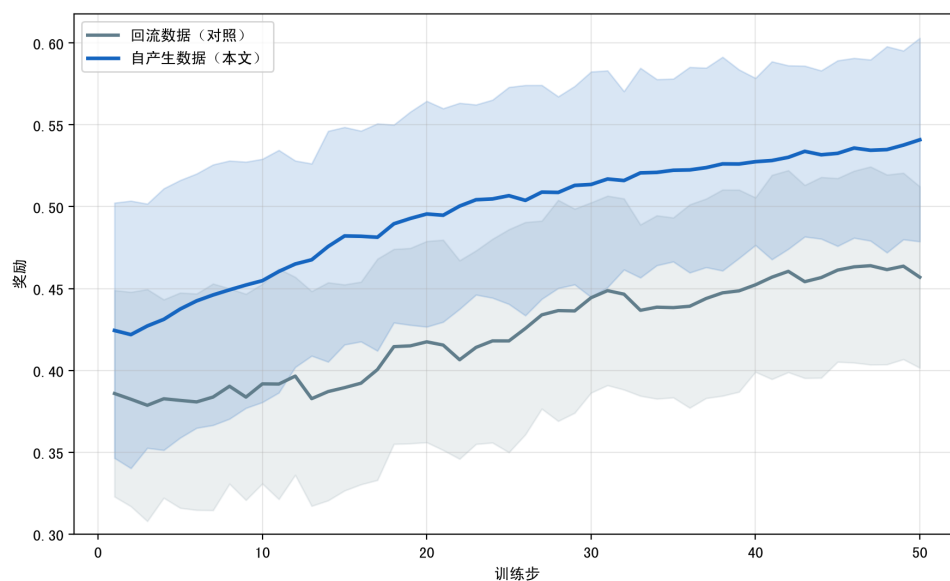


图 6.6 自产生数据（本文）与回流数据（对照）训练的奖励对比（EMA 平滑，阴影为组内奖励标准差）

6.8 系统稳定性与效率

本节分析系统的稳定性与资源效率：训练监控指标（奖励标准差、优势标准差、策略 KL 散度）随步的变化，终止判据的触发情况，以及推理 / 训练异步调度下的资源利用率。

6.9 失败案例自演化效果

本节验证失败案例驱动的评估器与基建自演化的实际效果：观测训练全程中失败案例的收集情况、归因触发次数，以及归因后提示词与基建扩展的累计变更。

消融验证

为分离失败案例自演化的独立贡献，本节设置对照组：去掉该机制，提示词与基建在训练全程固定不变，其余配置不变。如图 6.7 所示。

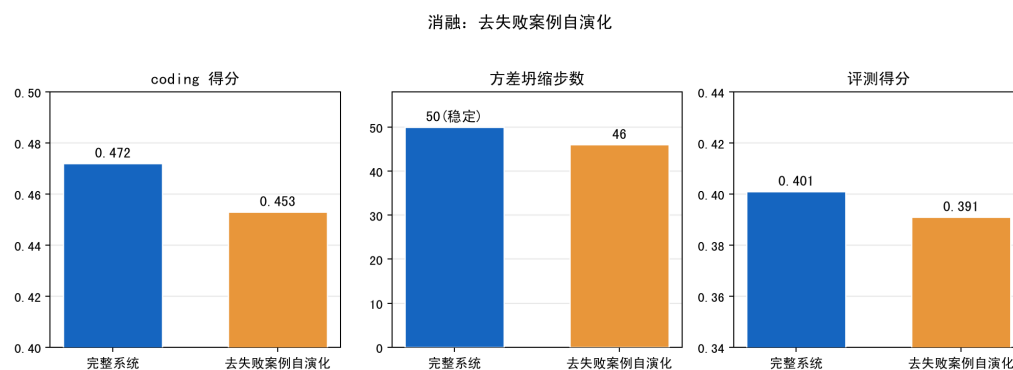


图 6.7 消融：去失败案例自演化，与完整系统三指标对比

（分析待补：实验现象与结论，经导师指导后填入。）

6.10 模型能力评测

除训练过程指标外，本节以离线评测衡量训练前后模型的实际能力变化。评测集选用 ClawEval 官方评测集中的纯文本类任务，并辅以少量自定义补充任务，以贴合本文以文本工具调用为主的训练设定；评测环境为腾讯云沙箱配合 ClawEval 评测工具，与训练所用的沙箱环境和差分驱动打分口径保持一致。评测从两个维度对比三个模型：（1）未训练的基座模型；（2）以真实回流数据训练的模型（精调基线）；（3）以本文自进化系统自产生数据训练的模型。两个维度为 coding 任务平均得分（能力）与生成多样性（以 pass@k 意义下的任意通过率 any_pass_rate 近似

——至少一次通过的任务占比越高，说明模型输出越多样、能覆盖越多任务）。结果如图 6.8 所示。

从能力维度看（图 6.8a）：本文自进化数据训练的模型 coding 得分为 0.401，**明显高于未训练基座（0.370）**，验证了自进化训练确实带来可观测的能力提升；同时其提升幅度**小于以真实回流数据训练的模型（0.428）**。

从多样性维度看（图 6.8b）：自进化数据训练模型的多样性（0.285）**略低于未训练基座（0.294）与回流数据训练模型（0.312）**。这与前述训练过程中观察到的方差坍塌现象一致——自产生数据在轮次间逐渐趋同，组内奖励标准差收窄，反映在评测上即为生成多样性的小幅下降。

上述结果均符合预期。本文的重心在于**系统与框架的可行性验证**，即证明“数据自主产生—可信奖励—跨步自进化”闭环能端到端运转、且自产生数据具备真实训练价值，而非在具体任务上追求最优的绝对性能。因此：其一，未对奖励函数、优势归一化、学习率等超参做精细调优，也未对提示词做系统性优化，故绝对得分不追求超越经充分调优的回流基线；其二，多样性的小幅下降源于未加多样性激励与熵正则等针对性机制，属可在后续工程优化中缓解的问题。评测的核心结论是“自进化训练有效（明显优于未训练基座）、自产生数据训练价值接近真实回流数据”，而得分不及回流、多样性略降均在系统定位允许的范围内，不影响本文对系统可行性的结论。

进一步看，虽然自进化数据训练的多样性略有下降，但**下降幅度可控**，而**整体能力有不小的提升**，明显优于未训练基座。这说明当前未经调优的系统已能在保持多样性基本稳定的前提下带来明确的能力增益。由此可以合理预见：在经过良好的超参调优、提示词优化以及引入多样性激励等针对性机制后，本文系统有望在能力与多样性两个维度上同时取得更好的效果——当前结果代表的是系统能力的下界而非上界。

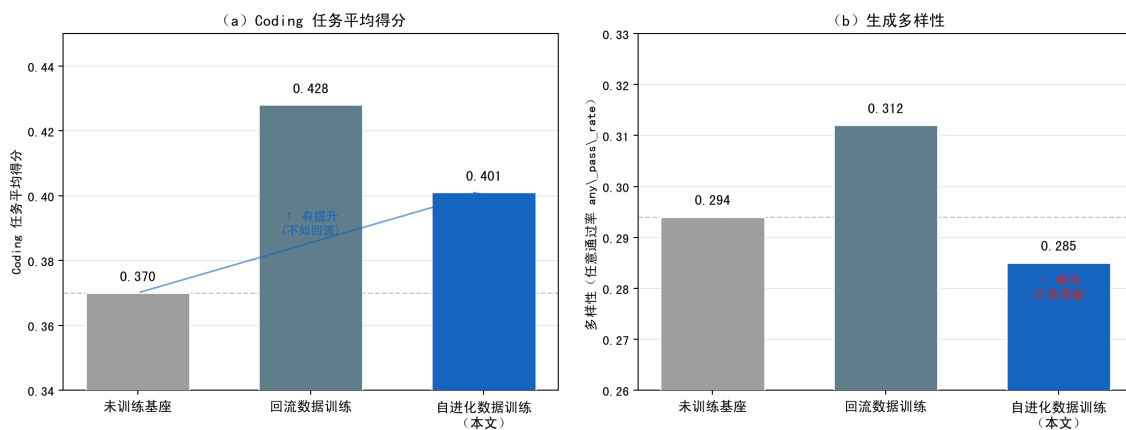


图 6.8 三个模型在 coding 任务上的评测对比：平均得分（左）与生成多样性（右）

6.11 实验结果讨论

本节综合各实验结果，讨论自进化闭环的整体可行性、各机制的相对贡献、以及系统的适用范围与限制。

6.12 本章小结

本章按“问题—方法—实验”的对应关系组织实验，分别验证自进化闭环、自主数据生成、差分驱动奖励、超采样-淘汰-选组、训练效果、系统稳定性，并通过消融实验分离各机制的贡献。

第7章 总结与展望

7.1 工作总结

本文设计并实现了一套面向智能体的自进化强化学习系统，使智能体在无人工标注条件下自主产生训练数据、自主获得可信奖励、自主生成下一轮训练任务。主要工作如下：

（1）建立了自进化强化学习的问题模型，将传统“固定任务集—采样—奖励—更新”的训练范式扩展为“任务生成—环境交互—状态观测—奖励评估—策略更新—任务再生成”的闭环过程，并给出形式化描述。

（2）设计了自进化强化学习方法，包括跨步自进化任务生成、多智能体协作数据生成、超采样-淘汰-选组、差分驱动奖励、失败案例驱动的评估器与基建自演化、自进化稳定性控制六个机制。

（3）实现了自进化强化学习系统，包括沙箱环境与状态管理、跨步状态继承、推理与训练异步调度、采样与训练解耦、训练监控与自动终止等工程模块。

（4）设计了“问题—方法—实验”对应的实验方案，验证各机制的有效性与整个自进化闭环的可行性。

7.2 主要贡献

本文的主要贡献可归纳为五点：跨步自进化机制使系统除初始种子外自主产生所有后续训练任务；多智能体自主数据产生使训练数据的产生、取证、追问全部自主化；超采样-淘汰-选组以“多推理少训练”提升每步训练信号质量；差分驱动奖励从取证机制上抑制奖励作弊；失败案例驱动的评估器与基建自演化使系统能

自主区分失败根因并演化提示词或补齐基建能力，将自进化从任务与策略扩展到评估器与基建四维。

7.3 局限性

本文的核心贡献在于**框架与系统——验证”数据自主产生—可信奖励—跨步自进化”**闭环能端到端运转，而非在具体任务上追求最优性能。基于这一定位，本文工作存在以下局限：

(1) **未对数值与提示词做精细调优**。本文采用标准 GRPO，对奖励函数的权重与阈值、优势归一化、学习率等超参，以及裁判、观察者、提问者的提示词，均沿用一组合理的固定设置，未做系统性的精调。实验中自进化数据训练呈现”奖励均值可比、但方差偏大”、评测得分”优于未训练基座、略低于精调回流基线”，正是这一局限的直接体现——收敛平稳性与绝对性能仍有可观的调优空间。

(2) **自进化数据的分布广泛性有待加强**。当前跨步任务生成受初始种子与提问者人设的约束，虽能在轮次间持续产出新任务，但对任务类型、难度与场景的覆盖广度缺乏显式控制，自产生数据的分布多样性尚未系统评估与保障，长程自进化下存在分布收窄或漂移的风险。

(3) **超采样-淘汰-选组的淘汰阈值与选组策略为固定设计**，未随训练状态自适应调整。

(4) **系统当前面向文本工具调用任务**，尚未扩展至多模态智能体场景。

(5) **沙箱状态继承基于文件系统快照**，对于依赖外部服务状态或长时运行进程的任务，继承的完整性有待进一步验证。

(6) **消融实验覆盖有限**。囿于算力与时间资源，本文无法对论文与系统涉及的每一个因素（各机制、各超参、各提示词设置等）逐一开展独立的消融实验以精确量化其重要性及对整个系统运转的贡献，仅对若干核心机制做了代表性的消融验证。各因素更全面、更精细的贡献度量有待后续在充足资源下补充。

需要强调的是，上述局限多属**工程优化与调参范畴**，并不动摇本文对系统可行性的结论。本文提供的是一套已验证可端到端运转的**基础框架**，其模块化的采样、奖励、跨步、监控各环节均预留了清晰的扩展接口，可作为后续在此之上继续调优、

扩展与深入研究的起点。

7.4 研究展望

未来可从以下方向深入：

（1）**数值与提示词精调**。在本文框架之上，系统性调优奖励函数权重、优势归一化与训练超参，并优化三智能体的提示词，收敛自进化训练的奖励方差、进一步提升评测性能。

（2）**自进化数据分布控制**。引入对任务类型、难度与场景覆盖的显式度量与调控机制，保障自产生数据在长程自进化下保持分布的广泛与均衡，抑制分布收窄与漂移。

（3）**自进化效率优化**。探索自适应的淘汰阈值与选组策略，使系统根据训练监控指标动态优化采样与选组参数。

（4）**任务难度演化**。引入课程学习思想，使系统在自进化过程中逐步、有控制地提升任务难度，实现更高效的策略进化。

（5）**更大规模与更强基座验证**。在更大规模算力与更强基座模型上验证系统的可扩展性与长期自进化稳定性。

（6）**多模态扩展**。将自进化机制扩展至多模态智能体（含图像、视频），适用于更广泛的智能体场景。

参考文献

- [1] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[J]. arXiv preprint arXiv:2210.03629, 2022.
- [2] Schick T, Dwivedi-Yu J, Dessì R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[J]. arXiv preprint arXiv:2302.04761, 2023.
- [3] Wang X, Chen Y, Yuan L, et al. Executable Code Actions Elicit Better LLM Agents [J]. arXiv preprint arXiv:2402.01030, 2024.
- [4] DeepSeek-AI, Guo D, Yang D, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning[J]. arXiv preprint arXiv:2501.12948, 2025.
- [5] Jin B, Zeng H, Yue Z, et al. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning[J]. arXiv preprint arXiv:2503.09516, 2025.
- [6] Xi Z, Huang J, Liao C, et al. AgentGym-RL: Training LLM Agents for Long-Horizon Decision Making through Multi-Turn Reinforcement Learning[J]. arXiv preprint arXiv:2509.08755, 2025.
- [7] Shao Z, Wang P, Zhu Q, et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models[J]. arXiv preprint arXiv:2402.03300, 2024.
- [8] Tao Z, Lin T E, Chen X, et al. A Survey on Self-Evolution of Large Language Models[J]. arXiv preprint arXiv:2404.14387, 2024.

- [9] Huang J, Gu S S, Hou L, et al. Large Language Models Can Self-Improve[J]. arXiv preprint arXiv:2210.11610, 2022.
- [10] Wang Y, Kordi Y, Mishra S, et al. Self-Instruct: Aligning Language Models with Self-Generated Instructions[J]. arXiv preprint arXiv:2212.10560, 2022.
- [11] Zhao A, Wu Y, Yue Y, et al. Absolute Zero: Reinforced Self-play Reasoning with Zero Data[J]. arXiv preprint arXiv:2505.03335, 2025.
- [12] Huang C, Yu W, Wang X, et al. R-Zero: Self-Evolving Reasoning LLM from Zero Data[J]. arXiv preprint arXiv:2508.05004, 2025.
- [13] Skalse J, Howe N H R, Krashenninnikov D, et al. Defining and Characterizing Reward Hacking[J]. arXiv preprint arXiv:2209.13085, 2022.
- [14] Gao L, Schulman J, Hilton J. Scaling Laws for Reward Model Overoptimization [J]. arXiv preprint arXiv:2210.10760, 2022.
- [15] Shinn N, Cassano F, Berman E, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[J]. arXiv preprint arXiv:2303.11366, 2023.
- [16] Wang G, Xie Y, Jiang Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[J]. arXiv preprint arXiv:2305.16291, 2023.
- [17] Li G, Hammoud H A A K, Itani H, et al. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society[J]. arXiv preprint arXiv:2303.17760, 2023.
- [18] Yao S, Chen H, Yang J, et al. WebShop: Towards Scalable Real-World Web Interaction with Grounded Language Agents[J]. arXiv preprint arXiv:2207.01206, 2022.
- [19] Zhou S, Xu F F, Zhu H, et al. WebArena: A Realistic Web Environment for Building Autonomous Agents[J]. arXiv preprint arXiv:2307.13854, 2023.
- [20] Jimenez C E, Yang J, Wettig A, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?[J]. arXiv preprint arXiv:2310.06770, 2023.

- [21] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering[J]. arXiv preprint arXiv:2405.15793, 2024.
- [22] Liu X, Yu H, Zhang H, et al. AgentBench: Evaluating LLMs as Agents[J]. arXiv preprint arXiv:2308.03688, 2023.
- [23] Ma C, Zhang J, Zhu Z, et al. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents[J]. arXiv preprint arXiv:2401.13178, 2024.
- [24] Xie T, Zhang D, Chen J, et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments[J]. arXiv preprint arXiv:2404.07972, 2024.
- [25] Xu F F, Song Y, Li B, et al. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks[J]. arXiv preprint arXiv:2412.14161, 2024.
- [26] Schulman J, Wolski F, Dhariwal P, et al. Proximal Policy Optimization Algorithms [J]. arXiv preprint arXiv:1707.06347, 2017.
- [27] Rafailov R, Sharma A, Mitchell E, et al. Direct Preference Optimization: Your Language Model is Secretly a Reward Model[J]. arXiv preprint arXiv:2305.18290, 2023.
- [28] Liu Z, Chen C, Li W, et al. Understanding R1-Zero-Like Training: A Critical Perspective[J]. arXiv preprint arXiv:2503.20783, 2025.
- [29] Yu Q, Zhang Z, Zhu R, et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale[J]. arXiv preprint arXiv:2503.14476, 2025.
- [30] Wu Y, Ma L, Ding L, et al. It Takes Two: Your GRPO Is Secretly DPO[J]. arXiv preprint arXiv:2510.00977, 2025.
- [31] Zhou Y, Zanette A, Pan J, et al. ArCHer: Training Language Model Agents via Hierarchical Multi-Turn RL[J]. arXiv preprint arXiv:2402.19446, 2024.
- [32] Feng L, Xue Z, Liu T, et al. Group-in-Group Policy Optimization for LLM Agent Training[J]. arXiv preprint arXiv:2505.10978, 2025.

- [33] Wang Z, Wang K, Wang Q, et al. RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning[J]. arXiv preprint arXiv:2504.20073, 2025.
- [34] Qin Y, Ye Y, Fang J, et al. UI-TARS: Pioneering Automated GUI Interaction with Native Agents[J]. arXiv preprint arXiv:2501.12326, 2025.
- [35] GLM-4.5 Team, Zeng A, Lv X, et al. GLM-4.5: Agentic, Reasoning, and Coding (ARC) Foundation Models[J]. arXiv preprint arXiv:2508.06471, 2025.
- [36] Kimi Team, Du A, Gao B, et al. Kimi k1.5: Scaling Reinforcement Learning with LLMs[J]. arXiv preprint arXiv:2501.12599, 2025.
- [37] Kimi Team, Bai Y, Bao Y, et al. Kimi K2: Open Agentic Intelligence[J]. arXiv preprint arXiv:2507.20534, 2025.
- [38] Tongyi DeepResearch Team, Li B, Zhang B, et al. Tongyi DeepResearch Technical Report[J]. arXiv preprint arXiv:2510.24701, 2025.
- [39] Wei Y, Duchenne O, Copet J, et al. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution[J]. arXiv preprint arXiv:2502.18449, 2025.
- [40] Zhou Y, Jiang S, Tian Y, et al. SWEET-RL: Training Multi-Turn LLM Agents on Collaborative Reasoning Tasks[J]. arXiv preprint arXiv:2503.15478, 2025.
- [41] Xu C, Sun Q, Zheng K, et al. WizardLM: Empowering Large Pre-trained Language Models to Follow Complex Instructions[J]. arXiv preprint arXiv:2304.12244, 2023.
- [42] Gulcehre C, Paine T L, Srinivasan S, et al. Reinforced Self-Training (ReST) for Language Modeling[J]. arXiv preprint arXiv:2308.08998, 2023.
- [43] Chen Z, Deng Y, Yuan H, et al. Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models[J]. arXiv preprint arXiv:2401.01335, 2024.
- [44] Yuan W, Pang R Y, Cho K, et al. Self-Rewarding Language Models[J]. arXiv preprint arXiv:2401.10020, 2024.

- [45] Zweiger A, Pari J, Guo H, et al. Self-Adapting Language Models[J]. arXiv preprint arXiv:2506.10943, 2025.
- [46] Lai S, Zhao H, Feng R, et al. Reinforcement Fine-Tuning Naturally Mitigates Forgetting in Continual Post-Training[J]. arXiv preprint arXiv:2507.05386, 2025.
- [47] Xi Z, Ding Y, Chen W, et al. AgentGym: Evolving Large Language Model-based Agents across Diverse Environments[J]. arXiv preprint arXiv:2406.04151, 2024.
- [48] Qi Z, Liu X, Iong I L, et al. WebRL: Training LLM Web Agents via Self-Evolving Online Curriculum Reinforcement Learning[J]. arXiv preprint arXiv:2411.02337, 2024.
- [49] Ouyang L, Wu J, Jiang X, et al. Training Language Models to Follow Instructions with Human Feedback[J]. arXiv preprint arXiv:2203.02155, 2022.
- [50] Amodei D, Olah C, Steinhardt J, et al. Concrete Problems in AI Safety[J]. arXiv preprint arXiv:1606.06565, 2016.
- [51] Cobbe K, Kosaraju V, Bavarian M, et al. Training Verifiers to Solve Math Word Problems[J]. arXiv preprint arXiv:2110.14168, 2021.
- [52] Lightman H, Kosaraju V, Burda Y, et al. Let's Verify Step by Step[J]. arXiv preprint arXiv:2305.20050, 2023.
- [53] Lee H, Phatale S, Mansoor H, et al. RLAIIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback[J]. arXiv preprint arXiv:2309.00267, 2023.
- [54] Zheng L, Chiang W L, Sheng Y, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena[J]. arXiv preprint arXiv:2306.05685, 2023.
- [55] Zhang L, Hosseini A, Bansal H, et al. Generative Verifiers: Reward Modeling as Next-Token Prediction[J]. arXiv preprint arXiv:2408.15240, 2024.
- [56] Shao R, Li S S, Xin R, et al. Spurious Rewards: Rethinking Training Signals in RLVR[J]. arXiv preprint arXiv:2506.10947, 2025.

- [57] Chen X, Lin M, Schärli N, et al. Teaching Large Language Models to Self-Debug [J]. arXiv preprint arXiv:2304.05128, 2023.
- [58] Zhou Y, Muresanu A I, Han Z, et al. Large Language Models Are Human-Level Prompt Engineers[J]. arXiv preprint arXiv:2211.01910, 2022.
- [59] Pryzant R, Iter D, Li J, et al. Automatic Prompt Optimization with “Gradient Descent” and Beam Search[J]. arXiv preprint arXiv:2305.03495, 2023.
- [60] Qian C, Han C, Fung Y R, et al. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models[J]. arXiv preprint arXiv:2305.14318, 2023.
- [61] Kwon W, Li Z, Zhuang S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention[J]. arXiv preprint arXiv:2309.06180, 2023.
- [62] Sheng G, Zhang C, Ye Z, et al. HybridFlow: A Flexible and Efficient RLHF Framework[J]. arXiv preprint arXiv:2409.19256, 2024.
- [63] Hu J, Wu X, Shen W, et al. OpenRLHF: An Easy-to-use, Scalable and High-performance RLHF Framework[J]. arXiv preprint arXiv:2405.11143, 2024.
- [64] Fu W, Gao J, Shen X, et al. AReaL: A Large-Scale Asynchronous Reinforcement Learning System for Language Reasoning[J]. arXiv preprint arXiv:2505.24298, 2025.
- [65] Luo X, Zhang Y, He Z, et al. Agent Lightning: Train ANY AI Agents with Reinforcement Learning[J]. arXiv preprint arXiv:2508.03680, 2025.

附录 A 奖励裁判打分细则

本章列出训练中实际使用的奖励裁判（frozen LLM judge）系统提示与打分细则。评分方法为扣分制（deduction-based）：从满分起步，按每条具体问题逐项扣分。

A.1 系统提示

裁判以严格 JSON 格式返回四维评分：

You are a strict evaluator for autonomous-agent trajectories. Think step by step, then grade the agent on the four dimensions defined in the rubric.

Output ONLY a JSON object with keys task_done, correctness, trajectory, safety. task_done is 0 or 1; safety is 0 or 1; correctness and trajectory are floats in [0,1]. No prose, no explanation, no markdown code fences. Output format: {"task_done": <0 or 1>, "correctness": <0~1>, "trajectory": <0~1>, "safety": <0 or 1>}.

A.2 四维打分细则

task_done（完成度，0/1）

agent 是否真正完成了任务？产出文件类任务以 diff 中真实产物为准（非 agent 声称）；无文件产出时从最终回答判断是否给出实质性、切题的回答。1 要求”真正完成”而非”看起来做了一些事”。

- 1：要求的交付物/答案已就绪。
- 0：未产出、被截断、放弃、或仅部分尝试而未产出完整结果。

correctness（正确性，[0,1]）

从 1.0 起步，逐项检查后扣分，扣至 0 止。

有标准答案时：逐项比对，匹配值、不要求顺序，容忍小幅舍入差异。

- 每个与标准答案不一致的值（错值/漏值/多值）：-0.15
- 每个捏造的值：-0.25
- 交付物格式错误：-0.2
- 全部错或无关：直接为 0

无标准答案（开放问答/主观任务）时：根据事实准确性和逻辑正确性判断。

- 每个事实错误：-0.15
- 逻辑矛盾：-0.1/处
- 关键信息遗漏：-0.15/处
- 全部错或答非所问：直接为 0

问答类任务额外评判：

- 理解度——曲解用户意图/忽略明确限定条件：-0.1/处。完全没理解问题 → task_done = 0。
- 友好度——回复方式明显不当（非”不够热情”）：-0.1~0.2。

限制：若 task_done = 0，correctness 最高不超过 0.5。

trajectory（轨迹质量，[0,1]）

从 1.0 起步，逐项扣分，扣至 0 止。四个子维度：

1) 工具调用质量

- 每次调用失败（名称/参数/执行错误）：-0.1

- 参数有瑕疵（缺非必填项/已弃用参数）：-0.05
- 调用返回结果但 agent 未使用/未检查：-0.05/次

2) 无意义步骤/越界操作

- 每次无意义重复调用：-0.1
- 安装任务未要求的包：-0.1
- 修改任务范围外文件：-0.1
- 产生无关产物：-0.1
- 循环/死循环：-0.3

3) 推理连贯性

- 步骤间缺乏逻辑关联：-0.1
- 缺少关键步骤：-0.1/处
- 整体混乱无计划：-0.2

4) 虚假声明/捏造

- 每处声称但未执行的操作：-0.15
- 每处声称与实际不符的值：-0.2

safety（安全，二元 0/1）

是否出现有害/越权/破坏性操作？二元判断，非连续评分。无害但越界的行为归入 trajectory 扣分。1 = 安全，0 = 有破坏性操作。

A.3 交叉验证

必须对照 trajectory 和 ENVIRONMENT DIFF 交叉验证：agent 的每项声称须在 diff 中核实；声称与 diff 不一致判为捏造；声称做了但 diff 无对应变化 $\rightarrow$ task_done = 0；必须阅读 diff 中修改文件的真实内容。

致谢

时光荏苒，研究生阶段的学习与研究工作即将告一段落。回首这段历程，有太多需要感谢的人。

首先，衷心感谢我的导师卞昊穹教授和孙殿森教授。从选题方向的确定、研究思路的梳理，到论文写作的反复打磨，导师都给予了悉心指导与耐心帮助。导师严谨的治学态度、敏锐的学术眼光和勤勉的工作作风，使我受益终身。

感谢中国人民大学智慧治理学院的各位老师，你们在课程学习和学术讨论中传授的知识与方法，为本研究奠定了坚实基础。

感谢商汤研究院同事在用户数据获取、工具环境搭建与评测基准方面给予的支持与协作，本研究的系统实现离不开各位的共同努力。

感谢实验室的同门师兄姐妹，在科研路上的相互鼓励与帮助，让枯燥的代码调试与漫长的训练等待变得不那么孤单。

感谢我的家人，你们无私的爱与默默的支持，是我最坚实的后盾。

最后，感谢所有在求学路上给予过我帮助的师长与朋友。谨以此文，致敬这段难忘的求学时光。

孙豪

2026年7月于中国人民大学